Operating System Support for Large-Scale Graph Learning

Course Report
Advanced Operating System

Shicheng Dai

225040523

School of Data Science
The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Graph Neural Networks (GNNs) extend deep learning from regular Euclidean inputs (images, text) to non-Euclidean graph inputs, and they have become one of the most important classes of models for node classification, link prediction, recommendation, fraud detection, and many other industrial applications. As the underlying graphs scale to billions of vertices and trillions of edges, however, the dominant cost of a GNN training iteration is no longer the matrix arithmetic of the GNN formula; it is the cost of moving data between storage, host memory, GPU memory, and remote devices, and of keeping irregular compute, communication and synchronisation aligned. In other words, large-scale GNN training has become a classical operating-system problem: caching, partitioning, scheduling, pipelining, compression and resource isolation.

This report studies three recent systems papers that each attack a different bottleneck in the large-scale GNN training pipeline:

- **BGL** [1] (NSDI 2023) targets sampling-based GNN training and rebuilds the front-end *data-supply pipeline* that feeds the GPU. It proposes a dynamic FIFO feature cache combined with a *proximity-aware ordering* of training nodes, a GNN-oriented graph partitioning algorithm that optimises both multi-hop locality and training-node balance, and a profiling-based resource-isolation scheme that allocates CPU/PCIe/network bandwidth to each pipeline stage.
- **MGG** [3] (OSDI 2023) targets intra-node execution and reorganises irregular multi-GPU GNN workloads as a *fine-grained dynamic software pipeline inside a GPU kernel*. It introduces a three-stage LR/LL/AC pipeline, a workload-balance-then-heterogeneity-then-granularity construction strategy, a warp-based mapping with an explicit "interleaving distance" knob, and an analytical-model-driven runtime that tunes the pipeline configuration across iterations.
- **AdaQP** [2] (MLSys 2023) targets distributed full-graph training and attacks *inter-device communication* directly. It stochastically quantises remote messages to low-precision integers with *adaptive* per-message bit-widths chosen by a variance-time bi-objective optimisation, splits each partition into a central and a marginal sub-graph so that central-node computation can be hidden inside marginal-node communication time, and isolates GPU resources via explicit CUDA-stream scheduling. A formal analysis proves an $O(T^{-1})$ convergence rate, identical to vanilla full-graph training.

The three systems are presented under a single pipeline view. As graph learning scales outward, the optimisation target also moves outward along the system stack: first *how data is prepared and supplied* (BGL), then *how the GPU actually executes* (MGG), and finally *what is exchanged between devices and when computation waits* (AdaQP). Each chapter reviews the motivation, the concrete design, and the reported experimental evidence of the corresponding paper, and a final chapter draws the cross-paper comparison. The report devotes one contribution chapter per paper, each organised as *Motivation* $\rightarrow$ *Design* $\rightarrow$ *Evaluation* $\rightarrow$ *Summary*.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Graph Neural Networks and the Systems Challenge

Graph Neural Networks generalise the idea of convolution to data that lives on graphs rather than on regular grids. At its most basic, message passing in a GNN produces a sequence of per-node embeddings by iteratively aggregating and transforming neighbourhood information. At layer $k+1$, the embedding $h_v^{(k+1)}$ of node v is computed from the embeddings of v's neighbours $N(v)$ at the previous layer via

$$a_v^{(k+1)} = \text{Aggregate}^{(k+1)}\big(\{h_u^{(k)} \mid u \in N(v)\} \cup \{h_v^{(k)}\}\big), \qquad h_v^{(k+1)} = \text{Update}^{(k+1)}\big(a_v^{(k+1)}\big).$$

The *Aggregate* step collects neighbour information; the *Update* step is a small, typically fully-connected neural network. Different GNN architectures differ mainly in the choice of these two functions. GCN uses a symmetric weighted mean, GraphSAGE allows a few different aggregators, GAT adds an attention mechanism, and GIN uses a sum plus a multilayer perceptron. Despite these algorithmic variations, they all share the same computational skeleton, and therefore the same dominant *systems* costs when the graph is large.

These costs are not abstract. A single graph in industry can have billions of nodes. The Pinterest graph reported by BGL [1] has roughly 2 B nodes and 17 B edges, requiring at least 18 TB of memory during training. On Ogbn-papers [1] (111 M nodes, 1.6 B edges) a three-hop neighbourhood sampled from one training node with fanout $\{15, 10, 5\}$ already produces a subgraph with roughly 400 K nodes and 195 MB of node features. If a worker is equipped with a 100 Gbps NIC and PCIe 3.0 x16, the feature-transfer rate saturates at about 60 mini-batches per second, yet the same worker's V100 GPU can compute 400 mini-batches per second on GraphSAGE [1]. The gap between the *feeding rate* and the *compute rate* is an order of magnitude. This is the fundamental reason why reported GPU utilisation of mainstream GNN frameworks on giant graphs is often as low as 5%–15%.

1.2 Three System-Critical Pressures

Three properties of large-scale GNN workloads make them particularly stressful for the underlying system:

1. **Neighbour explosion and irregular access.** Each additional layer multiplies the number of neighbours that must be fetched. Because the adjacency lists are sparse and irregular, the access pattern has almost no spatial locality and poor cache efficiency, in sharp contrast to the dense, regular accesses of CNNs or Transformers.
2. **Cross-layer data dependency.** A GNN layer cannot finish its local compute until all required neighbour features or embeddings have arrived. This creates a hard synchronisation dependence between communication/IO and compute at every layer.
3. **Communication-dominated scaling.** When training is scaled to many GPUs or many machines, the fraction of time spent in cross-device message exchange grows faster than the fraction spent in pure arithmetic. Beyond a few GPUs, the critical path is no longer compute but data movement and waiting, a phenomenon repeatedly demonstrated in all three papers covered by this report.

The central observation, which frames the rest of the report, is that the real systems problem of large-scale GNN training is not the GNN formula itself but *how the system aligns data, communication, and compute at scale*.

1.3 Two Training Regimes

Modern GNN systems fall into two families that differ fundamentally in *which data ever reaches the GPU*.

Sampling-based (mini-batch) training. Only a sampled subgraph enters the GPU each iteration. The training loop is effectively a pipeline consisting of (i) subgraph sampling on the distributed graph store, (ii) feature retrieval for sampled nodes, and (iii) GNN model computation on the worker GPU. The performance-critical stages are sampling and feature retrieval, which can collectively account for 80%–90% of the per-iteration time in DGL or Euler on large graphs [1]. The representative paper in this family covered by this report is **BGL**.

Full-graph distributed training. The entire graph remains logically active during training. Each device owns a partition and exchanges messages with the others at every layer of every iteration. The performance-critical issues are remote message volume, workload imbalance (stragglers), and waiting time under synchronous communication. The representative papers in this family covered by this report are **MGG** (for a single multi-GPU node) and **AdaQP** (for multiple machines).

1.4 Contributions of This Report

This report does not propose new algorithms. Its contribution is a structured, OS-flavoured reading of three recent systems papers on large-scale graph learning:

1. A *unified pipeline view* of GNN training under which the three papers are understood as optimisations of three different stages: data preparation (BGL), intra-node execution (MGG), and inter-device communication (AdaQP).
2. A *per-paper analysis* that follows the same sub-structure—motivation, design, evaluation, summary—so that the three systems can be compared at matching levels of granularity.
3. A *cross-paper synthesis* that shows how, as graph learning scales outward, the optimisation target moves outward along the classical operating-system stack, and how the three papers reuse well-known OS primitives (cache, partition, pipeline, compression, resource isolation) adapted to the irregular nature of graph workloads.

1.5 Organisation

Chapter 2 introduces the GNN training pipeline and locates the three bottleneck stages that the three papers target. Chapter 3 studies BGL; Chapter 4 studies MGG; Chapter 5 studies AdaQP. Each paper chapter is organised as *Motivation* $\rightarrow$ *Design* $\rightarrow$ *Evaluation* $\rightarrow$ *Summary*. Chapter 6 draws a cross-paper comparison and concludes.

Chapter 2

Background and Related Work

2.1 A Training Iteration as a Three-Stage Pipeline

A GNN training iteration on a large graph can be viewed as a pipeline with three system-critical stages:

(A) Data preparation.
The graph is partitioned across storage servers; samplers draw subgraphs on demand; a feature-retrieving component fetches the features of nodes touched by each subgraph and pushes them to the GPU. This stage dominates end-to-end training time in sampling-based training on very large graphs. For example, on Ogbn-papers with one GPU worker, DGL and Euler spend 82% and 87% of per-iteration time on this stage respectively [1].

(B) Intra-node execution.
On each node, the GNN kernel runs over the subgraph or the partition. In multi-GPU nodes this stage interleaves local computation with remote-neighbour access. It is particularly difficult to schedule because GNN workloads are irregular and sparse, unlike the dense regular workloads for which existing multi-GPU frameworks were designed [3].

(C) Inter-device communication.
Devices exchange features, embeddings and embedding gradients at every layer. In synchronous distributed full-graph training, communication accounts for 66%–78% of per-epoch time on representative datasets, and the share grows with the number of partitions because the remote-neighbour ratio grows [2].

Table 2.1 shows how the three papers map onto this pipeline view.

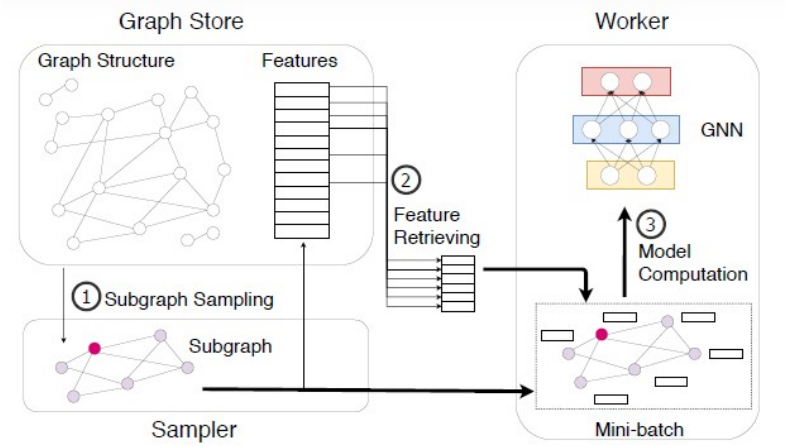


Figure 2.1: Sampling-based GNN training pipeline: subgraph sampling on graph store servers, feature retrieving from storage to the GPU, and GNN model computation on the worker.

Table 2.1: Mapping of the three papers to the three training stages.

Pipeline stage	Representative paper in this report
(A) Data preparation — sampling, partitioning, feature retrieving	BGL [1]
(B) Intra-node execution — irregular multi-GPU kernel scheduling	MGG [3]
(C) Inter-device communication — message volume, imbalance, waiting	AdaQP [2]

2.2 Why This Framing is Useful

If BGL, MGG, and AdaQP were presented as three unrelated optimisation tricks, the reader would miss the reason why they make such different design choices. The pipeline framing exposes that reason:

- BGL is *data-centric*. Its job is to feed the GPU. Its design space is dominated by caching, graph partitioning and resource isolation, i.e. the classical OS concerns on the IO path.
- MGG is *kernel-centric*. Its job is to keep irregular local/remote operations packed into a warp-level pipeline inside the GPU. Its design space is pipeline construction, warp-to-SM mapping, and asynchronous memory primitives.
- AdaQP is *communication-centric*. Its job is to reduce the volume of inter-device messages and to hide the residual waiting time. Its design space is lossy message encoding, central/marginal scheduling, and explicit CUDA-stream isolation.

2.3 OS-Style Techniques Recycled for GNNs

None of the three papers invents a new mathematical primitive for GNNs. Instead, each recycles classical OS techniques and adapts them to the irregular, communication-heavy nature of graph learning: caching with locality-aware replacement (BGL's FIFO+proximity); multi-level partitioning with locality and balance constraints (BGL's block coarsening, MGG's local/remote neighbour split); fine-grained software pipelining (MGG's LR/LL/AC pipeline); lossy compression (AdaQP's stochastic integer quantisation); and resource-isolation-based scheduling (BGL's profiled bandwidth allocation, AdaQP's CUDA-stream ordering). This is why the three systems form a natural study unit for an operating systems course: they show classical OS ideas being re-applied under the very specific constraints of irregular, sparse, communication-heavy workloads.

Chapter 3

BGL: Feeding the GPU via a Dedicated Data-Supply Pipeline

3.1 Motivation

3.1.1 Sampling-Based GNN Training and its Bottlenecks

BGL [1] targets *sampling-based* GNN training, in which each training iteration consists of three stages: (i) subgraph sampling from a distributed graph store; (ii) feature retrieval of the sampled nodes from storage servers to the GPU; and (iii) forward/backward computation of the GNN model on the GPU. BGL refers to the first two stages collectively as *Data I/O and Preprocessing*.

On giant graphs these two stages dominate end-to-end time. The paper measures on Ogbn-papers that DGL and Euler spend 82% and 87% of per-iteration time on data I/O and preprocessing, and the corresponding maximum GPU utilisation is only 15% and 5% [1]. The root cause is a throughput mismatch: a V100 GPU can compute a GraphSAGE mini-batch in about 20 ms (so a single DGX node could run roughly 400 mini-batches per second), but saturating the 100 Gbps NIC and PCIe 3.0 x16 bus with neighbour-expanded subgraphs only yields about 60 mini-batches per second. Pipelining helps a little, but the imbalance between the stages is so large that the GPU remains starved.

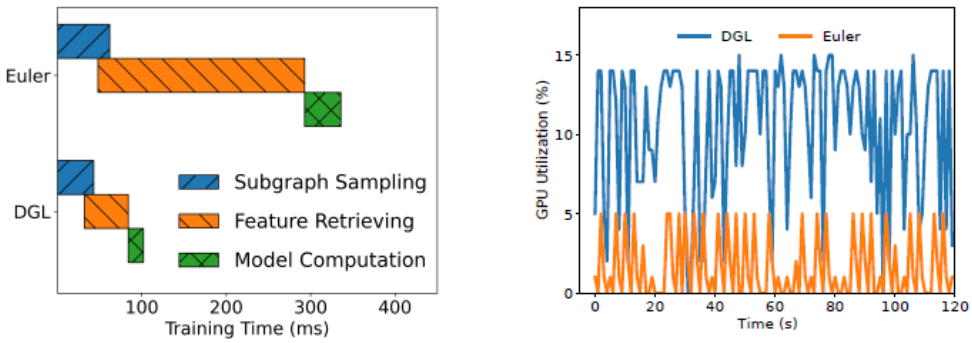


Figure 3.1: Per-iteration time breakdown of DGL and Euler on Ogbn-papers, and the corresponding GPU utilisation. Data I/O and preprocessing dominate; GPU utilisation drops to 5–15%.

3.1.2 Three Concrete Challenges

BGL identifies three concrete challenges that must be solved simultaneously to remove the bottleneck.

Challenge 1: Ineffective caching for feature retrieval. A natural idea is to cache "hot" features using the power-law degree distribution of real graphs. PaGraph [1] adopts a *static* cache populated with the top-degree nodes before training and never updated. On giant graphs this is too rigid: when only 10% of nodes fit in the cache, the static hit ratio drops below 40%. But *dynamic* cache policies are expensive in practice. The paper reports that best-effort implementations of classical $O(1)$ LRU and LFU cost about 80 ms per batch to update, which is already four times the GPU compute time for one mini-batch and therefore unacceptable.

Challenge 2: A partitioner that is scalable, multi-hop locality-aware, and training-node balanced. The second bottleneck is cross-partition sampling traffic. Existing partitioners either ignore structure (random) or optimise only the wrong thing. METIS and most state-of-the-art partitioners (GMiner, CuSP) minimise one-hop edge-cut and balance the number of *all* nodes per partition. But GNN samplers

draw multi-hop neighbourhoods, so only multi-hop locality actually reduces sampling traffic; and only about 10% of nodes are training nodes, so balancing all nodes may leave the *training* workload badly unbalanced. Moreover, some classical partitioners (METIS, PaGraph) have prohibitive memory or time complexity on billion-node graphs [1].

Challenge 3: Contending preprocessing stages. GNN preprocessing is much more complex than DNN preprocessing. Sampling, graph serialisation, feature retrieval, and the cache engine all compete for CPU cores, memory bandwidth, PCIe bandwidth, and network bandwidth. Frameworks that let all stages compete freely (DGL, PyG, Euler) or that defer scheduling to TensorFlow/PyTorch produce unpredictable slowdowns: one fast stage simply pushes the bottleneck onto another. A systematic resource assignment is needed.

3.2 Design

3.2.1 Architecture

BGL is a distributed pipeline (see Figure 3.2) whose components are:

- A *graph partition module* that runs once on the raw graph data in HDFS and produces disjoint partitions for later loading.
- A set of *graph store servers*, each holding one partition in memory and co-locating a *sampler* on CPU.
- *Worker machines*, each hosting one or more GPU workers with a shared CPU/GPU *feature cache engine*.
- A *cross-machine parameter synchronisation* path for the GNN model parameters.

The architectural commitment is that sampling and feature retrieval are not collapsed into the training worker: they are first-class system components with their own resources and their own place in the pipeline. This is what makes profiling-based resource isolation (Challenge 3) meaningful at all.

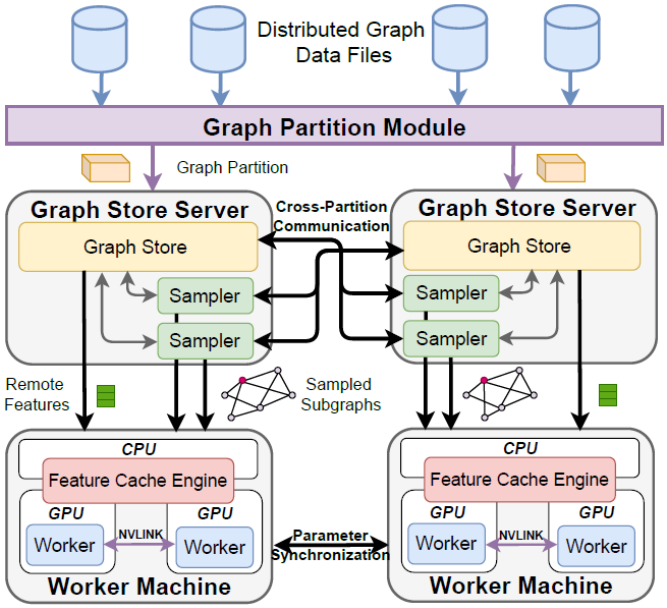


Figure 3.2: Architecture of BGL: distributed graph store servers (each holding a partition and a sampler), worker machines with a shared CPU/GPU feature cache engine, and a parameter synchronisation path.

3.2.2 Dynamic Feature Cache Engine

Why FIFO, and why it is not enough on its own

BGL's first observation is that the cost of classical replacement policies dominates the benefit. FIFO's update cost is below 20 ms per batch, which barely fits inside a GPU iteration budget, whereas LRU and LFU cost about 80 ms. The catch is that, without further help, FIFO achieves a *lower* hit ratio than even the static policy, because it evicts hot and cold features at the same rate.

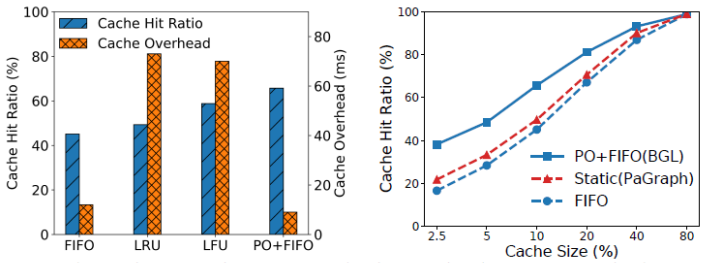
Proximity-Aware Ordering (PO)

To restore FIFO's hit ratio, BGL co-designs the cache *policy* with the sampling *order* of training nodes. The key observation is that nearby mini-batches draw multi-hop neighbourhoods from training nodes, and if consecutive training nodes are close in the graph, their neighbourhoods overlap—so the features already cached from one mini-batch are likely to be re-requested soon by the next. BGL therefore performs *BFS traversal of the training nodes*: starting from random BFS roots, it generates a BFS sequence of training nodes and then draws mini-batches in that order. This maximises temporal locality.

A naive pure-BFS order, however, violates the i.i.d. assumption of SGD and damages model accuracy. BGL keeps BFS but reintroduces controlled randomness in two ways: (i) it maintains many BFS sequences, each from a random root, and constructs each mini-batch by a round-robin pick across sequences; (ii) each BFS sequence is circularly shifted by a random offset so that deterministic tail effects (the many small connected components of giant graphs tend to pile up at the end of a BFS) are smeared across batches.

The number of BFS sequences is chosen using a *shuffling-error* bound from Meng et al.: if the total-variation distance ε between the output distribution of an ordering algorithm and the uniform distribution is at most $\sqrt{bM/n}$ (b=batch size, M=#workers, n=training-set size), convergence is not affected. BGL therefore generates many BFS sequences, estimates the shuffling error from the label distribution, and increases the number of sequences until the bound is satisfied. The cost of this procedure is under 1% of training time in practice [1].

As measured in the paper, FIFO+PO dominates both static caching and plain FIFO across cache sizes from 2.5% to 80% of graph size, and keeps update overhead below 20 ms per batch.



(a) Trade-off between hit ratios and overhead (10% cache size).

(b) Cache hit ratios with different cache sizes.

Figure 3.3: (a) Trade-off between cache hit ratio and per-batch update overhead for static caching, LRU, LFU and FIFO. (b) Cache hit ratio of static caching, plain FIFO and FIFO+PO for cache sizes from 2.5% to 80% of graph size.

Multi-GPU and CPU Two-Level Cache

GNN models are small (about 100 MB of parameters) while modern servers hold hundreds of gigabytes of CPU memory and tens of gigabytes of GPU HBM. BGL therefore uses *all* of this memory as cache:

- Each GPU has its own *cache map* (a hashmap from node ID to a slot index) and *cache buffer* (the actual feature tensors). To avoid duplication across GPUs, node IDs are assigned to GPUs by a modulo operation, so each GPU owns a disjoint slice of the cache.

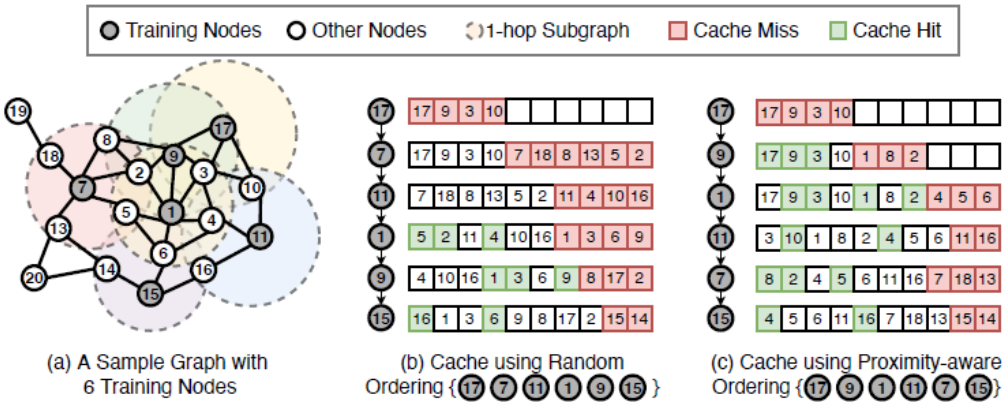


Figure 3.4: Proximity-aware ordering: training nodes are reordered along multi-source BFS sequences (with random shifts and round-robin batching) so that consecutive mini-batches share multi-hop neighbours.

- GPU-to-GPU fetches go over NVLink via P2P memory copy. Without NVLink, BGL's throughput drops by 50% because inter-GPU fetches must cross PCIe at much lower bandwidth.
- On top of the multi-GPU cache, a CPU cache with the same FIFO+PO policy further enlarges effective cache capacity and reduces graph-store traffic.

The cache *workflow* is designed to avoid lock contention. A naive per-slot lock forces CUDA synchronisation and costs a lot. BGL instead queues all operations to a given GPU cache and lets exactly one *processing thread* poll that queue; that thread is the only reader and writer of the corresponding cache buffer. This reduces the overhead by $8\times$ compared with locks.

3.2.3 GNN-Oriented Graph Partitioning

Workflow: Coarsen, Assign, Uncoarsen

BGL's partitioner is a three-step distributed algorithm:

1. **Multi-level coarsening.** Block generators load disjoint HDFS shards and run a multi-source BFS, broadcasting block IDs from random root nodes until a block reaches a threshold size (e.g., 100K). Unlike METIS's maximal matching, BFS-based coarsening preserves multi-hop connectivity. Because real graphs have many small connected components, BGL then performs a second round that merges small blocks into their large neighbours (or randomly if no large neighbour exists), further reducing the coarsened-graph size.
2. **Block collection and assignment.** A block assigner collects all blocks and greedily assigns each to a partition using the heuristic below.
3. **Uncoarsening.** Block generators map the assignment back to individual nodes and save the partition to HDFS.

With E_1 edges in the BFS-coarsened graph and E_2 edges after merging, and j hops of locality considered, the paper shows the total time complexity is $O(|E| + |E_1| + j|E_2|)$, which is dominated by the linear scan of $|E|$ and is therefore friendly to billion-node graphs.

The Assignment Heuristic

Each block B is assigned to the partition $P(i)$ that maximises

$$\max_{i\in[k]}\left\{\left(\sum_j |P(i)\cap\Gamma_j(B)|\right)\cdot\left(1-\frac{|T(i)|}{C_T}\right)\cdot\left(1-\frac{|P(i)|}{C}\right)\right\}, \tag{3.1}$$

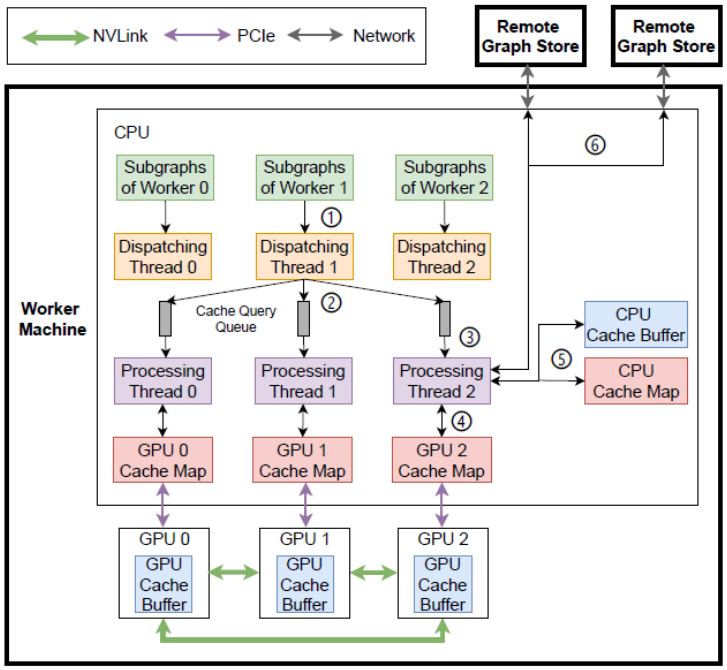


Figure 3.5: Structure and workflow of the multi-GPU+CPU feature cache engine: per-GPU cache map and cache buffer, queue-polled processing thread, NVLink P2P fetches between GPUs, CPU-side cache fallback.

where $\Gamma_j(B)$ is the set of j-hop neighbour blocks of B; $T(i)$ and $P(i)$ denote the sets of training nodes and all nodes already assigned to partition i; $C_T = |T|/k$ and $C = |V|/k$ are per-partition capacity constraints. The three factors encode, respectively, multi-hop locality, training-node balance, and overall node balance. The product enforces them jointly: a block goes to the partition that is both close in the multi-hop sense *and* not overfull in either count.

3.2.4 Resource Isolation for Contending Stages

BGL splits training into eight asynchronous pipeline stages (sampling, subgraph construction, subgraph transfer, subgraph processing, cache workflow, feature transfer, model computation, and parameter synchronisation). Some of these compete on the same physical resources:

- Sampling and subgraph construction contend for CPU on the graph store servers.
- Subgraph processing and the cache workflow contend for CPU on the worker machines.
- Subgraph movement and feature movement contend for PCIe bandwidth.

The paper reports a specific pathology: the cache stage saturates around 40 cores and even *degrades* beyond 64 because of OpenMP synchronisation and memory-bandwidth limits. Leaving the stages to fight each other therefore gives much worse performance than a profiled, deliberately rationed allocation.

BGL's allocator profiles each stage and solves

$$\min \max \left\{ \frac{T_1}{c_1},\ \frac{T_2}{c_2},\ T_{\text{net}},\ \frac{T_3}{c_3},\ \frac{D_I}{b_I},\ f(c_4),\ \frac{D_{II}}{b_{II}},\ T_{\text{gpu}} \right\}$$

subject to $c_1{+}c_2 \le C_{\text{gs}}$, $c_3{+}c_4 \le C_{\text{wm}}$, and $b_I{+}b_{II} \le B_{\text{pcie}}$, i.e. minimise the slowest stage subject to CPU and PCIe caps. All per-stage times T_i, data sizes D_I, D_{II} and the fit function $f(c_4) = a/c_4 + d$ for the non-linear cache stage are profiled. A bounded brute-force search finds the allocation in under 20 ms on average. The result is that BGL converts resource contention from an emergent problem into an explicit optimisation problem.

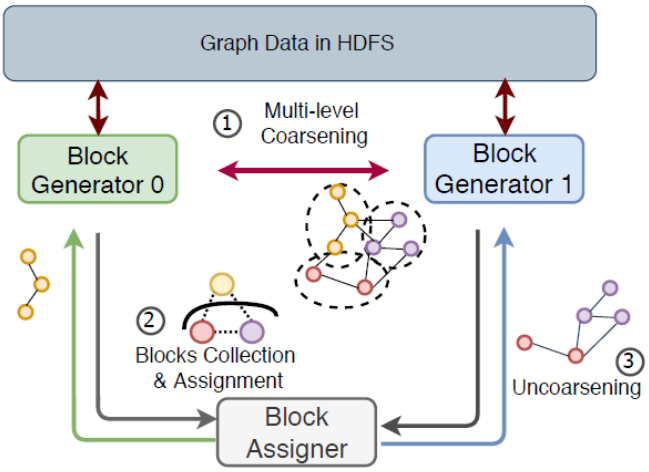


Figure 3.6: Partition workflow: (1) multi-level coarsening, where block generators run BFS to form blocks; (2) block collection and greedy assignment; (3) uncoarsening back to nodes.

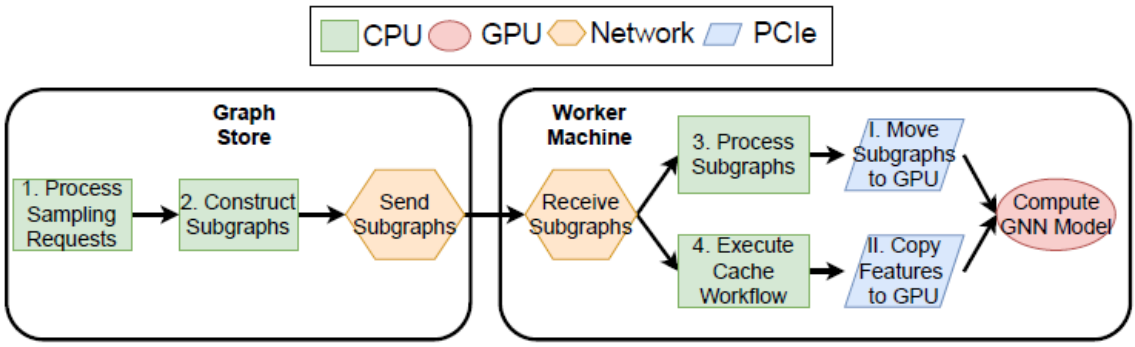


Figure 3.7: Eight-stage asynchronous pipeline of BGL, with the contention relations among sampling, subgraph processing, the cache workflow, and PCIe transfers.

3.3 Evaluation

3.3.1 Setup

BGL is evaluated on a cluster of 4 GPU servers (each with 8 V100 GPUs connected by NVLink v2, 96 vCPU, 356 GB RAM) and 32 CPU servers (96 vCPU, 480 GB RAM), all connected by 100 Gbps NICs. Three datasets are used (Table 3.1); three GNN models (GCN, GAT, GraphSAGE) are compared against four baselines: Euler, DGL (DistDGL), PyG, and PaGraph [1].

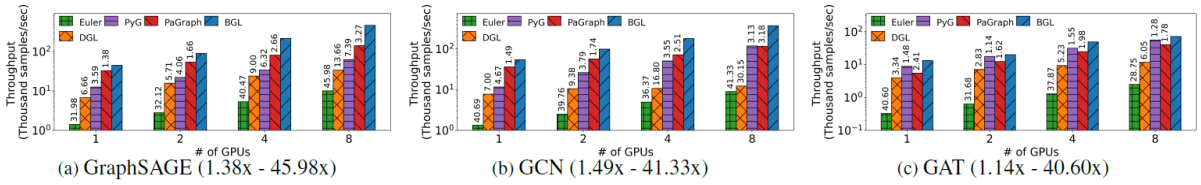
Table 3.1: Evaluation datasets used by BGL [1].

Dataset	#Nodes	#Edges	#Features	#Classes
Ogbn-products	2.4 M	123 M	100	47
Ogbn-papers	111.1 M	1.6 B	128	172
User-Item	1.2 B	13.7 B	—	—

3.3.2 End-to-End Speedup

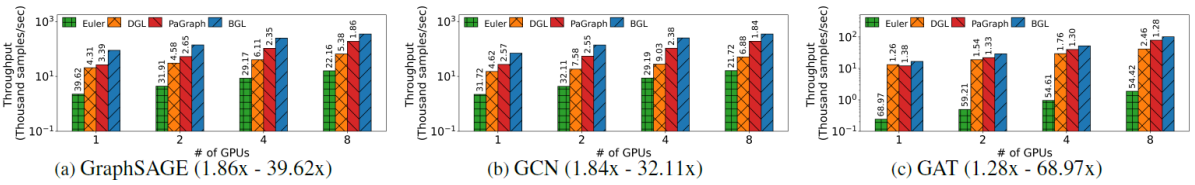
Across all combinations of dataset, model, and number of GPUs from 1 to 8, BGL delivers $1.14\times$–$69\times$ speedups over the baselines. Among baselines, PaGraph is the strongest (static GPU cache); even so

BGL beats PaGraph by up to 3.27× with a geometric-mean speedup of 1.91×. Over PyG, DGL, and Euler the geometric means are 3.02×, 7.04×, and 20.68× respectively. Gains are larger on GraphSAGE and GCN (up to 30×) than on GAT (up to 8×), because GAT's attention mechanism makes compute a larger fraction of the iteration and leaves less "waiting" for BGL to remove.



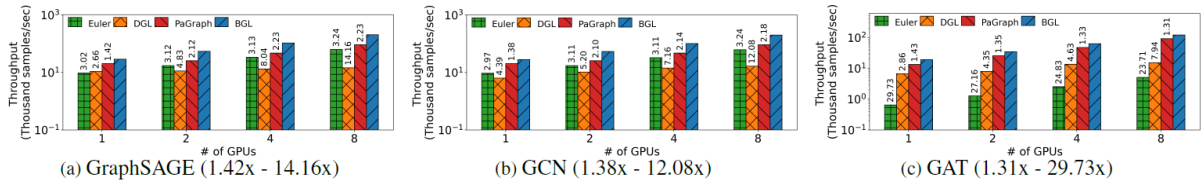
(a) GraphSAGE (1.38x - 45.98x) (b) GCN (1.49x - 41.33x) (c) GAT (1.14x - 40.60x)

Figure 3.8: Training throughput on Ogbn-products for GCN, GAT, and GraphSAGE under 1–8 GPUs, comparing BGL with Euler, DGL, PyG and PaGraph.



(a) GraphSAGE (1.86x - 39.62x) (b) GCN (1.84x - 32.11x) (c) GAT (1.28x - 68.97x)

Figure 3.9: Training throughput on Ogbn-papers.



(a) GraphSAGE (1.42x - 14.16x) (b) GCN (1.38x - 12.08x) (c) GAT (1.31x - 29.73x)

Figure 3.10: Training throughput on User-Item (1.2 B nodes, 13.7 B edges).

3.3.3 GPU Utilisation and Scalability

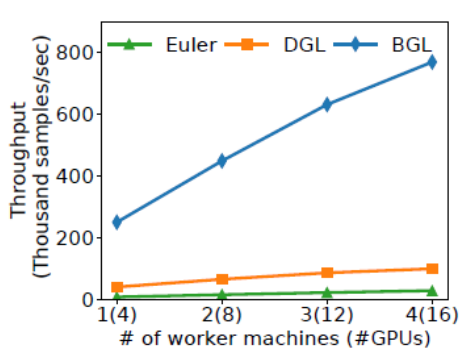
On Ogbn-products with 8 GPUs, BGL raises GPU utilisation from DGL's 38% to 99% on GAT, and from 10% to 65% on GraphSAGE. Scaling from 1 to 8 GPUs, DGL only achieves about 3× throughput (bottlenecked by PCIe), whereas BGL scales almost linearly thanks to its NVLink-served multi-GPU cache. Scaling from 1 to 4 worker machines (4 GPUs each), BGL achieves 76% of linear scaling.

3.3.4 Mechanism-Level Evidence

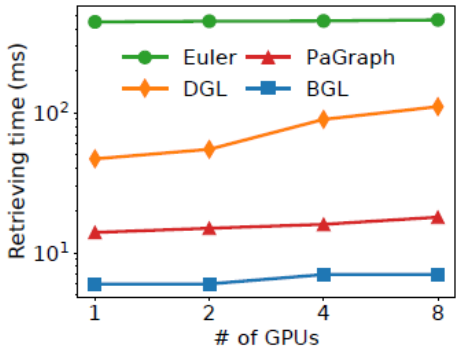
Ablation experiments decompose the speedup across mechanisms. Feature retrieval time per mini-batch on Ogbn-papers drops by 98%, 88% and 57% versus Euler, DGL, and PaGraph respectively, validating the FIFO+PO cache. Sampling time per epoch drops by at least 20% over random partitioning and 14%/10% over GMiner on Ogbn-products/Ogbn-papers, thanks to multi-hop-aware partitioning; the ratio of cross-partition sampling communication drops by 25%/44%/33% on the three datasets. For resource isolation, BGL without isolation is actually *slower* than PaGraph on Ogbn-products because the stages fight each other; BGL with isolation is up to 2.7× faster than the naive version [1].

3.3.5 Model Quality

The paper runs 100-epoch convergence comparisons with GAT and GraphSAGE on all three datasets. BGL (using PO) converges to the same accuracy as DGL (using random ordering) on every configuration

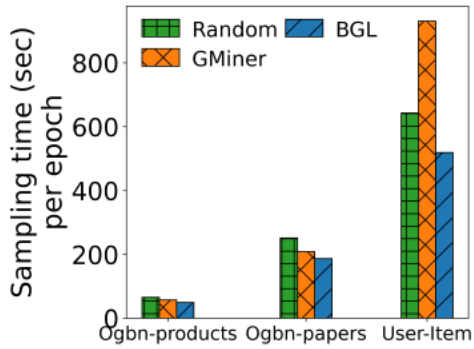


(a) Scalability to multiple workers

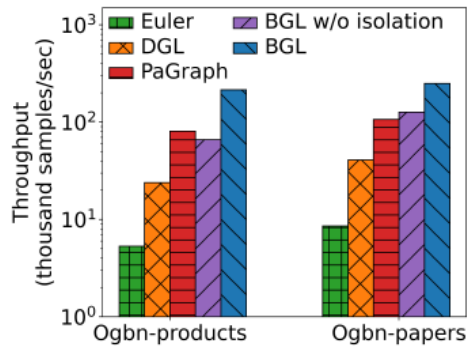


(b) Feature retrieving time per mini-batch

Figure 3.11: Scalability and feature-retrieval performance of BGL. (a) Scalability of BGL to multiple worker machines (one to four workers, 4 GPUs each); (b) feature retrieving time per mini-batch on Ogbn-papers, comparing Euler, DGL, PaGraph and BGL. Reproduced from BGL [1], Figures 18 and 13.



(a) Sampling time and cross-partition communication ratio



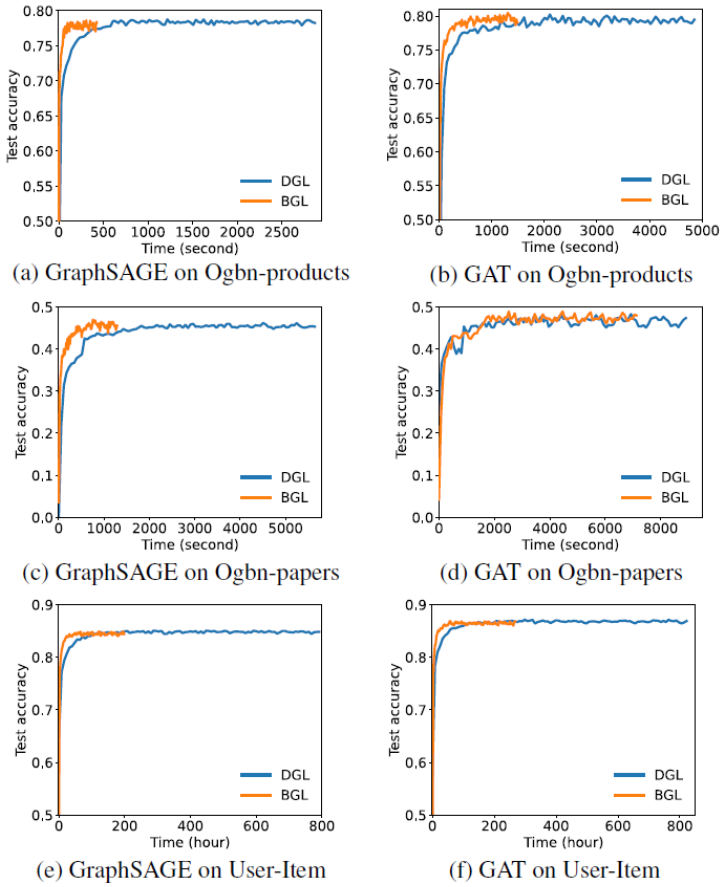
(b) Throughput with vs. without resource isolation

Figure 3.12: Effect of GNN-oriented partitioning and resource isolation in BGL. (a) Sampling time per epoch and the ratio of cross-partition sampling communication, for random partitioning, GMiner, and BGL's GNN-oriented partitioning; (b) throughput with vs. without resource isolation — without isolation BGL can be slower than PaGraph, while with isolation it is up to 2.7× faster than its non-isolated variant. Reproduced from BGL [1], Figures 14, 15 and 17.

but reaches that accuracy much sooner in wall-clock time. This confirms that BGL's speedups are purely system-level and do not trade accuracy for speed.

3.4 Summary

BGL represents the "feed the GPU better" line of OS support for graph learning. Its contribution is to recognise that, in sampling-based training, the bottleneck is a *front-end data-supply pipeline*, and to design three OS-style mechanisms that together keep that pipeline balanced: a dynamic FIFO+PO feature cache backed by a multi-GPU+CPU two-level buffer; a GNN-oriented partitioner that optimises multi-hop locality and training-node balance via the heuristic of Eq. (3.1); and a profiling-based resource allocator that minimises the slowest pipeline stage. The combination raises GPU utilisation from 5%–15% to up to 99% and delivers geometric-mean speedups of 1.9× over the strongest baseline on real web-scale graphs.



(a) GraphSAGE on Ogbn-products (b) GAT on Ogbn-products

(c) GraphSAGE on Ogbn-papers (d) GAT on Ogbn-papers

(e) GraphSAGE on User-Item (f) GAT on User-Item

Figure 3.13: Validation accuracy vs. wall-clock time for GAT and GraphSAGE under DGL (random ordering) and BGL (PO). BGL reaches the same accuracy strictly earlier.

Chapter 4

MGG: Fine-Grained Intra-Kernel Pipelining on Multi-GPU

4.1 Motivation

4.1.1 Why Existing Multi-GPU GNN Systems Fall Short

If BGL optimises the pipeline *before* data reaches the GPU, MGG [3] goes one step deeper: it asks how irregular GNN execution should be organised *inside* a multi-GPU node. Existing multi-GPU GNN systems (e.g., DGL, ROC, NeuGraph) treat communication and computation as two separate phases, exactly as dense DNN systems do. This separation is adequate for dense regular workloads such as AllReduce-synchronised DNN layers, but it is inappropriate for GNNs: the aggregation is sparse and irregular; remote-neighbour access is fine-grained (tens of bytes per embedding); and the dependency between a node's local compute and the arrival of its remote neighbours is fine-grained too. Consequently, the critical path of multi-GPU GNN execution is inter-GPU communication, and any coarse-grained phase separation wastes cycles that *could* be overlapped.

4.1.2 Three Observations That Enable Intra-Kernel Pipelining

MGG's design is based on three observations:

1. **GNN workloads expose partial dependencies.** Each node's neighbour embeddings can be aggregated sequentially or in parallel with proper synchronisation, and different operations on a GPU naturally compete for SMs at runtime. These two kinds of dependency open fine-grained overlap opportunities between *different nodes'* local computation and *other nodes'* remote access.
2. **GPUs are already designed for pipelining.** Modern GPUs schedule many logical processing units (threads, warps, blocks) on a small number of physical SMs. The same hardware mechanism that amortises unit compute cost can, if we co-run computation and communication warps, also amortise unit communication cost and fill in the idle cycles.
3. **NVSHMEM enables fine-grained, GPU-initiated inter-GPU communication.** NVSHMEM's warp-level `nvshmem_float_warp_get` primitive lets warps issue tens-of-bytes remote reads without CPU involvement, and naturally coalesces the requests. It is the right primitive to build an intra-kernel pipeline; UVM is too coarse-grained (KB-level page migration) and incurs CPU page-fault handling.

Together these observations motivate the central view of MGG: *multi-GPU GNN execution is a fine-grained dynamic software pipeline*. "Fine-grained" means each stage is one neighbour's aggregation; "dynamic" means the pipeline structure varies with input graph and hardware.

4.2 Design

The MGG system has three coordinated components (Figure 4.1): a GNN-tailored pipeline construction module, a GPU-aware pipeline mapping module, and an intelligent runtime parameter optimiser. This section presents each in turn.

4.2.1 GNN-Tailored Pipeline Construction

A Three-Stage LR/LL/AC Pipeline

MGG models a neighbour-aggregation workload as a three-stage pipeline:

LR : loading *remote* neighbour embeddings,

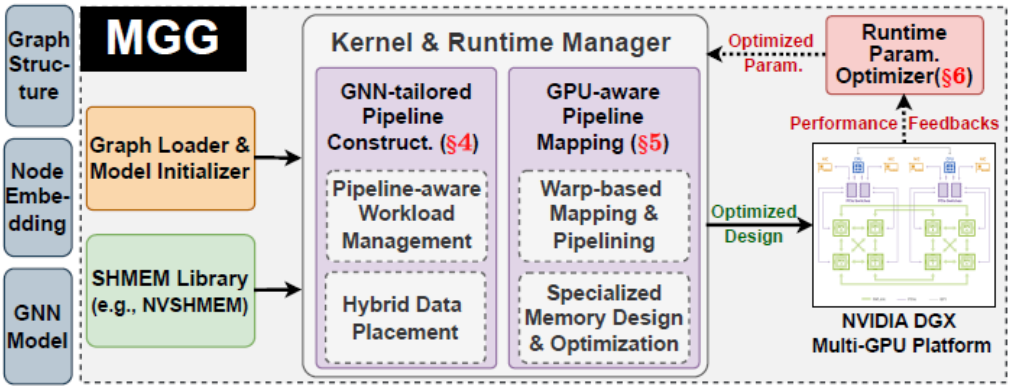


Figure 4.1: Overview of MGG: kernel & runtime manager (GNN-tailored pipeline construction and GPU-aware pipeline mapping), runtime parameter optimiser with performance feedback, and the NVSHMEM-based multi-GPU back end.

LL : loading *local* neighbour embeddings,

AC : aggregation *computation*.

A remote-neighbour aggregation uses LR+AC; a local-neighbour aggregation uses LL+AC. This three-phase abstraction is general: GCN has a low local-to-remote ratio, GAT has a higher one, sparse graphs have a higher remote-to-local ratio. Different inputs and different GNNs are simply different mixes of the two basic pathways.

Three-Step Pipeline-Aware Workload Management

Building an efficient pipeline out of the raw workload is done in three steps.

Step 1 — Workload-aware inter-GPU balancing. A range-constrained binary search (Algorithm 1 of the paper, reproduced in 1) splits the node pointer array of the graph so that each GPU gets approximately the same number of *edges*. Each per-GPU chunk then becomes one pipeline, and its workload is grouped by node into mixed local+remote partitions. This eliminates the first source of inefficiency: an unbalanced chunk would put an entire long-latency critical path on one pipeline.

Algorithm 1 Range-constrained binary search for edge balance (MGG).

Require: node pointer array nPtr, edge list eList, number of GPUs G
Ensure: list of $G-1$ split points on nodes
1: $ePerGPU \leftarrow (|\text{eList}| + G - 1)/G$
2: $lastPos \leftarrow 0$
3: **for** $s = 0$ to $G-1$ **do**
4: $\quad nid \leftarrow \text{binSearch}(\text{nPtr}, ePerGPU, lastPos, n)$
5: $\quad lastPos \leftarrow nid$; append nid to output list
6: **end for**

Step 2 — Heterogeneity-aware pipeline-bubble reduction. Inside each pipeline, the LL and LR stages have very different latencies, so a naive single mixed workload still produces bubbles. MGG splits each chunk into two CSRs: a *local* CSR and a *remote* CSR. Aggregation runs on the two CSRs separately and synchronises at the end. On all-to-all-connected DGX platforms the cost of accessing any remote GPU is roughly equal, which justifies treating "remote" as a single class. The local/remote split moves the pipeline from a state with many bubbles to a state where LR is densely overlapped with LL+AC.

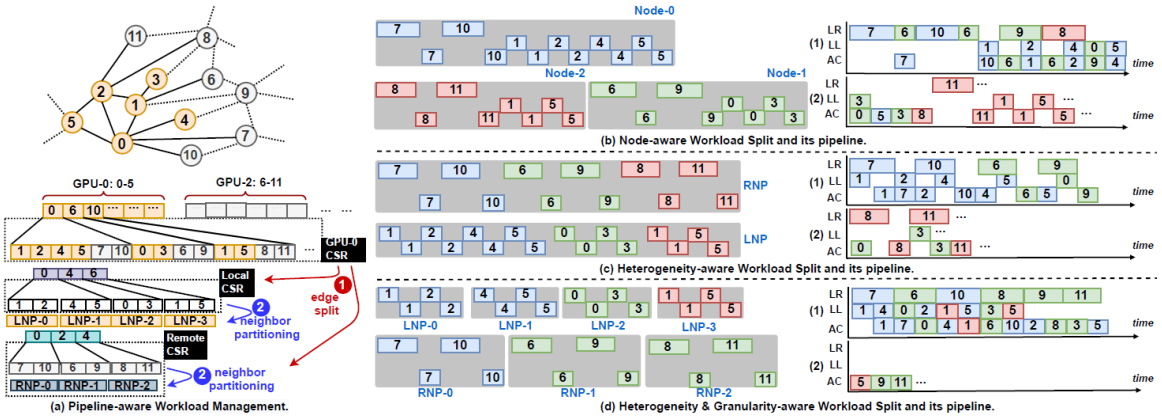


Figure 4.2: Pipeline-aware workload management. (a) Local-remote workload split into LNP and RNP partitions. (b) Node-aware split (raw pipeline with bubbles). (c) Heterogeneity-aware split. (d) Granularity-aware split (additional fixed-size neighbour partitioning).

Step 3 — Granularity-aware intra-GPU enhancement. Within each CSR, nodes have very different numbers of neighbours, so mapping one node per warp would again imbalance work. MGG therefore partitions each node's neighbour list into fixed-size pieces ("neighbour partitions") of ps neighbours each. A smaller ps gives more parallelism and better load balance but adds synchronisation overhead (atomic reductions into the target node) inside the warp. The trade-off is exposed as the parameter ps and tuned by the runtime (§4.2.3).

Hybrid GNN Data Placement

Pipeline construction is only effective if the data layout lets each stage get data at the right speed from the right place. MGG splits GNN data across two memory spaces:

- *Node embeddings (NEs)*, which are large (high-dimensional) and need to be reachable from every GPU, are placed in *NVSHMEM-shared global memory*, partitioned equally across GPUs. Because shared global memory offers approximately equal access speed from every GPU, remote workloads are evenly costed, which is what the pipeline requires.
- *Graph partition (GP) structure*, which is small and scalar (node pointer and edge lists), is kept in each GPU's *private* DRAM. This avoids paying inter-GPU cost on tiny pointer chases. An address translation unit maps local rebased indices to the correct remote GPU when needed.

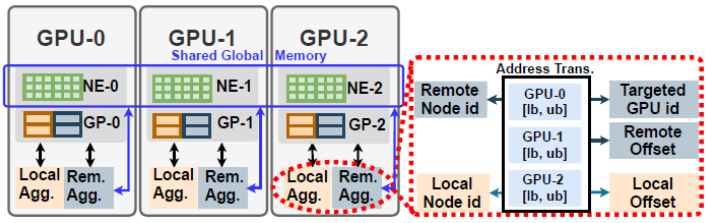


Figure 4.3: Hybrid storage layout and communication pattern: NEs in NVSHMEM-shared global memory equally partitioned across GPUs; GP structure in private GPU global memory; address-translation unit maps rebased indices.

4.2.2 GPU-Aware Pipeline Mapping

Warps as the Scheduling Unit

MGG picks the *warp* (32 threads) as the unit that handles one neighbour partition. Threads inside a warp cooperatively pull different dimensions of an embedding; smaller than a warp would waste parallelism, larger than a warp would add synchronisation across sub-blocks. Warp-level NVSHMEM primitives coalesce remote reads into a single remote transaction, amortising per-transaction overhead. The paper's comparison of NVSHMEM primitives (Fig. 10(c)) confirms warp-level is fastest on average, with thread-level losing coalescing and block-level paying block synchronisation cost.

Workload Interleaving and the "dist" Knob

If consecutive warps handle the same type of partition (all remote or all local), SMs assigned only remote-heavy warps suffer much longer latency than those with only local-heavy warps, so the SMs imbalance again. MGG *interleaves* local and remote partitions along the warp ID axis, controlled by an *interleaving distance dist*: every *dist* local partitions are followed by *dist* remote partitions. A larger *dist* gives SMs a longer uniform run; a smaller *dist* gives finer-grained overlap. This parameter is searched at runtime.

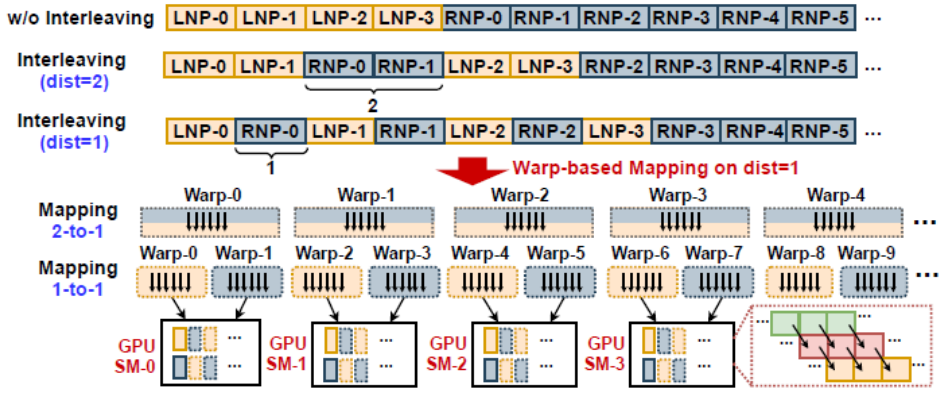


Figure 4.4: Warp-based mapping with workload interleaving. Top: without interleaving, warps with consecutive IDs handle the same type of partition and skew SM load. Middle/Bottom: interleaving with $dist = 1$ and $dist = 2$.

Specialised Shared Memory and Async Primitives

MGG lays out each thread block's shared memory into three contiguous regions: neighbour IDs, partial aggregation results, and remotely-fetched embeddings. The size depends on warps per block (*wpb*) and embedding dimension D and is passed to the kernel at launch.

For intra-warp overlap, MGG replaces synchronous remote reads with *asynchronous* NVSHMEM remote reads. Each warp *simultaneously* launches its local load and initiates the remote load, then does local aggregation while remote data is in flight; once remote data arrives, it runs remote aggregation and loops onto the next pair. Profiling shows that without this, the remote access dominates $\sim 60\%$ of warp latency, so intra-warp overlap is essential to hide it.

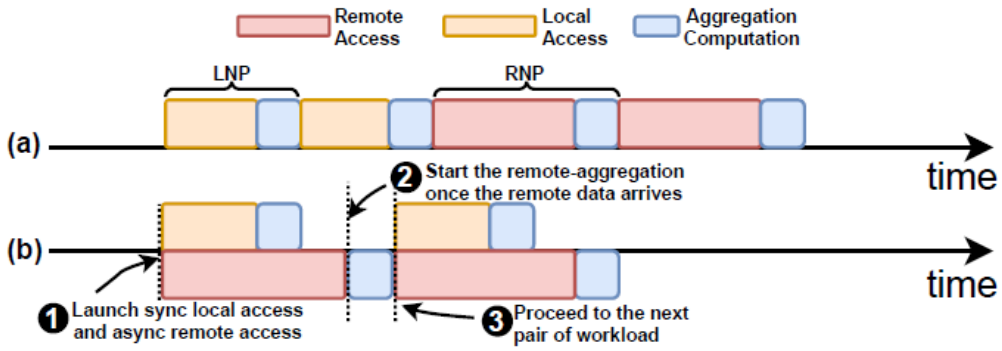


Figure 4.5: Intra-warp overlap of computation and remote access. (a) Synchronous remote access yields a long stall after each LL. (b) Asynchronous remote access overlaps remote load with the local aggregation of the previous LNP.

4.2.3 Intelligent Runtime Design

Analytical Model

MGG models two key quantities as functions of three parameters (ps, *dist*, *wpb*):

$$WPW = 2 \cdot ps \cdot D \cdot dist,$$
$$SMEM = ps \cdot wpb \cdot \text{IntS} + 2 \cdot wpb \cdot D \cdot \text{FloatS},$$

with IntS $=$ FloatS $= 4$ B on GPUs. From ps and *dist* it computes the number of warps, then the number of thread blocks and the blocks per SM. Hardware constraints (≤ 108 SMs on A100, ≤ 164 KB shared memory per SM) bound the feasible region, giving the ranges $ps \in [1, 32]$, $dist \in [1, 16]$, $wpb \in [1, 16]$.

Cross-Iteration Heuristic Search

The runtime initialises $ps = dist = wpb = 1$ and performs a greedy coordinate search in the order $ps \rightarrow dist \rightarrow wpb$. First ps is increased until latency stops improving (coarse algorithmic granularity). Then *dist* is increased until latency stops improving (pipeline overlap). Finally *wpb* is increased; if increasing *wpb* degrades latency, the algorithm back-tracks ps to its second-best value and retries. The rationale for this order is both *spatial* (coarse $\rightarrow$ medium $\rightarrow$ fine) and *temporal* (ps decided at load time, *dist* at kernel init, *wpb* at runtime). A lookup table records latency per iteration, and the best configuration is applied. The whole search typically converges in about 10 iterations.

4.3 Evaluation

4.3.1 Setup

Implementation: $\sim$ 9K LoC, CUDA 11.2, NVSHMEM 2.0.3, cuDNN 8.2. Platform: an NVIDIA DGX-A100 (dual AMD Rome 7742, 1 TB host RAM, 8$\times$A100-40GB over NVSwitch, 600 GB/s bi-directional). Two models are used: GCN (2 layers, 16 hidden) and GIN (5 layers, 64 hidden); seven graph datasets are used (Table 4.1). Baselines are DGL (with PyTorch-Direct zero-copy), MGG-UVM (MGG's pipeline with UVM instead of NVSHMEM), and ROC (Legion runtime, learning-based partitioning).

4.3.2 End-to-End Results

Versus DGL. MGG achieves average speedups of $4.25\times$ (GCN) and $4.57\times$ (GIN) across the seven datasets. The speedup grows with the number of GPUs ($4 \rightarrow 8$), confirming that MGG shines precisely when inter-GPU pressure grows. GraphSAGE and GAT, when added, show speedups of 1.76–$7.05\times$ and 1.62–$3.39\times$ respectively over DGL (Table 4.2).

Table 4.1: Evaluation datasets used by MGG [3].

Dataset (abbr.)	#Nodes	#Edges	#Dim	#Class
reddit (RDD)	232,965	114,615,892	602	41
enwiki-2013 (ENWIKI)	4,203,323	202,623,226	300	12
it-2004 (IT04)	41,291,594	1,150,725,437	256	64
ogbn-paper100M (PAPER)	111,059,956	1,615,685,872	128	64
ogbn-products (PROD)	2,449,029	61,859,140	100	47
ogbn-proteins (PROT)	132,534	39,561,252	8	112
com-orkut (ORKT)	3,072,441	117,185,083	128	32

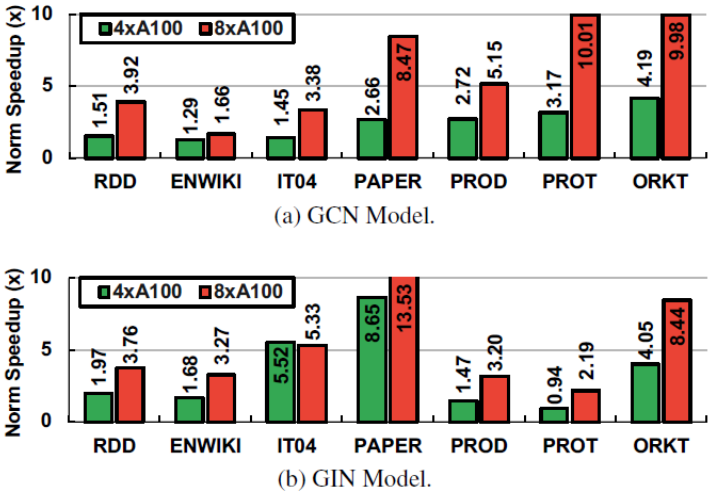


Figure 4.6: Speedup of MGG over DGL on (a) GCN and (b) GIN, with 4 and 8 A100 GPUs, across seven datasets.

Versus MGG-UVM. Replacing NVSHMEM with UVM in the same MGG design still leaves MGG at $4.58\times$ / $5.04\times$ (GCN/GIN average) ahead. The explanation is that UVM's page-fault-based remote access has a $\sim$ 4 KB granularity while node embeddings are under 0.4 KB, so embeddings frequently span two pages and each transfer pays multiple page faults. This illustrates that the pipeline is only useful if the communication primitive is actually fine-grained.

Versus ROC. On $8\times$A100, MGG achieves $12.30\times$ / $9.35\times$ average speedups over ROC on GCN/GIN, with the largest gains on the largest graphs (IT04, PAPER). ROC's Legion runtime uses DMA for bulky transfers, which gives it high throughput but poor latency, and its communication-computation separation leaves no overlap opportunity for MGG's strategy to exploit.

4.3.3 Mechanism-Level Evidence

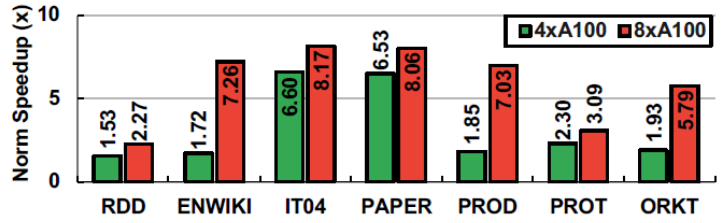
Neighbour partitioning. Disabling it increases latency by an average of $2.26\times$ because warp-level imbalance dominates, especially on graphs with heavy remote access.

Workload interleaving. Disabling it (mapping remote and local partitions to disjoint warp ranges) increases latency by an average of $1.89\times$ because skewed warp scheduling re-creates the bubbles the pipeline was designed to remove.

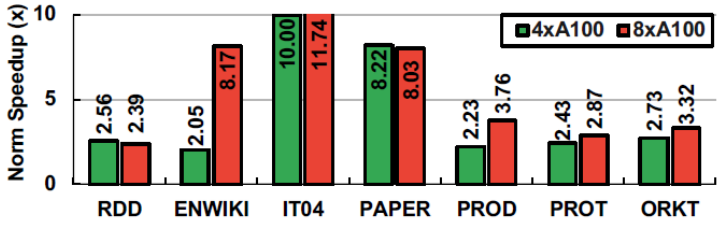
Primitive choice. Warp-level NVSHMEM consistently beats thread-level (no coalescing) and block-level (extra synchronisation) across datasets.

Table 4.2: MGG speedup over DGL on GraphSAGE and GAT (8×A100) [3].

Model	RDD	ENWIKI	IT04	PAPER	PROD	PROT	ORKT
SAGE	4.97×	1.76×	1.99×	3.53×	7.05×	3.39×	3.53×
GAT	2.65×	1.62×	2.06×	3.04×	2.06×	3.39×	3.04×



(a) GCN Model.



(b) GIN Model.

Figure 4.7: Speedup of MGG over MGG-UVM on (a) GCN and (b) GIN, with 4 and 8 A100 GPUs.

Runtime parameter search. The greedy $ps \rightarrow dist \rightarrow wpb$ search converges in about ten iterations and reduces latency by up to 68% versus the initial configuration. Over many training iterations this alone is a significant fraction of wall-clock gains, and it is what makes MGG portable across input and hardware settings.

SM utilisation. Versus MGG-UVM, MGG raises Achieved Occupancy by 39.20% on average and SM utilisation by 21.15% on average. These low-level metrics confirm that the intra-kernel pipeline is indeed packing the SMs better.

4.3.4 Generality

The paper also shows that MGG's design applies to deep-learning recommendation models (DLRM) with sparse embedding lookup. Using MGG's fine-grained remote access in place of NCCL, DLRM embedding lookup time drops from 315.27 ms to 119.66 ms (2.64×, see Table 4.3). This indicates that the "irregular sparse remote access" problem MGG solves is not unique to GNNs.

Table 4.3: DLRM embedding-lookup time, original implementation vs. MGG-accelerated, on Criteo Kaggle with 4 GPUs [3].

Implementation	DLRM (original, NCCL)	DLRM (MGG)
Time (ms)	315.27	119.66

4.4 Summary

MGG represents the "make irregular GNN execution schedulable inside the GPU kernel" line of OS support. Its contribution is to turn the fuzzy notion of communication-computation overlap into a concrete

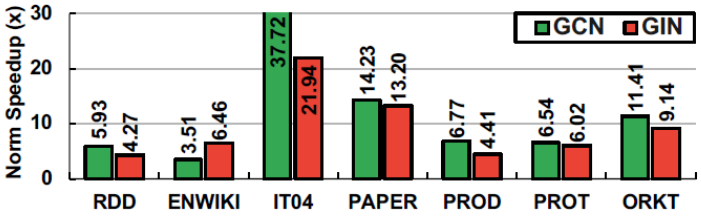


Figure 4.8: Speedup of MGG over ROC for GCN and GIN with 8×A100.

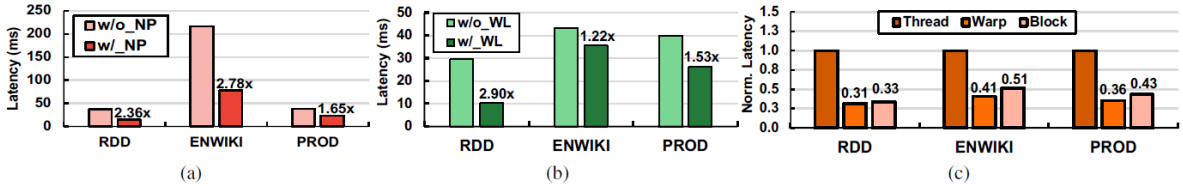


Figure 4.9: Optimisation analysis on RDD/ENWIKI/PROD with 4×A100. (a) With vs. without neighbour partitioning. (b) With vs. without workload interleaving. (c) Choice of NVSHMEM primitive (thread / warp / block).

three-stage LR/LL/AC software pipeline and to layer three construction steps (balance → heterogeneity → granularity), a warp-based mapping with an explicit interleaving knob, an intra-warp asynchronous pipeline, and an analytical-model-driven runtime on top of it. The combination delivers geometric-mean multi-× speedups over DGL, MGG-UVM and ROC on both GCN and GIN, and extends to DLRM, showing that fine-grained intra-kernel pipelining is a useful general-purpose technique for sparse-irregular multi-GPU workloads.

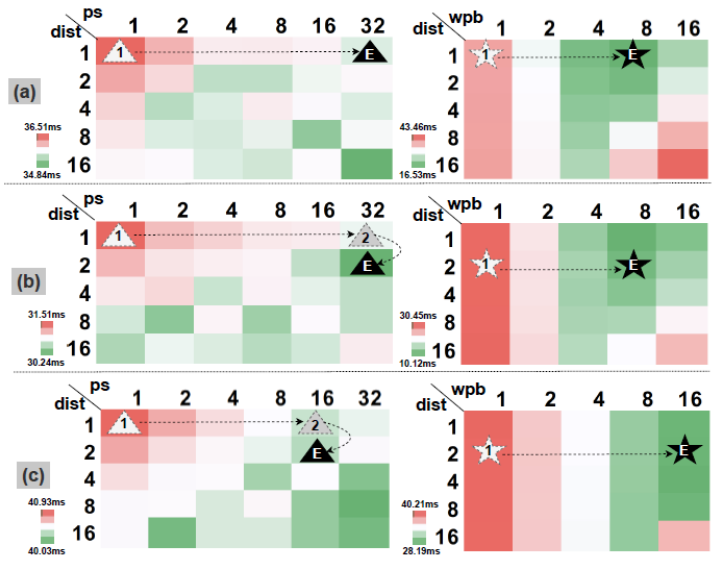


Figure 4.10: Parameter selection for three settings: (a) RDD on 4×A100, (b) RDD on 8×A100, (c) RDD on 4×V100. The triangle marks the optimal $(ps, dist)$ found by the heuristic; the star marks the best *wpb*.

Chapter 5

AdaQP: Adaptive Message Quantization and Central/Marginal Parallelization

5.1 Motivation

5.1.1 Communication Dominance in Distributed Full-Graph Training

AdaQP [2] pushes the optimisation target one step further outward. Where MGG scheduled local and remote accesses *inside* a single multi-GPU node, AdaQP attacks *inter-device* communication in distributed full-graph training that crosses machines.

In vanilla distributed full-graph training ("Vanilla" in the paper), the graph is partitioned by METIS and each GNN layer's forward and backward pass performs an all-to-all exchange of node features, embeddings, and embedding gradients (collectively called *messages*) for remote neighbours. The paper's Table 1 shows that on a three-layer GCN the communication cost is 66.78% of per-epoch time on Reddit (2 machines, 1 GPU each), 75.2% on Reddit (2M-2D), 75.59% on Ogbn-products (2M-2D), and 78.22% on AmazonProducts (2M-4D). The share *grows* with the number of partitions because the remote-neighbour ratio grows: e.g., $41.5\% \rightarrow 62.6\%$ on Reddit when moving from 2M-1D to 2M-2D.

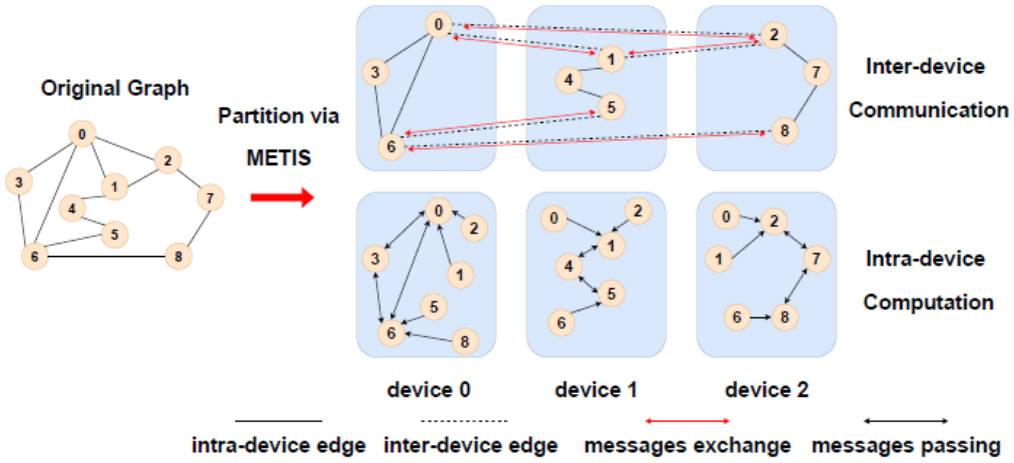


Figure 5.1: Vanilla distributed full-graph training: an input graph partitioned by METIS across three devices, with intra-device computation interleaved with inter-device communication of messages for remote neighbours.

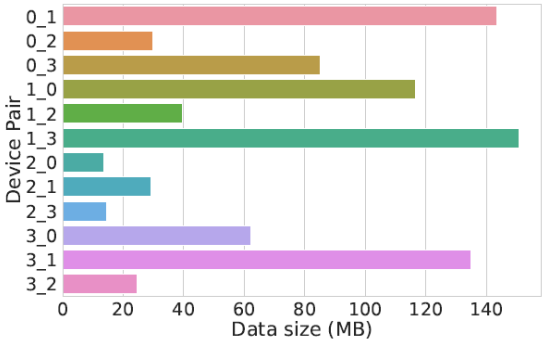
Table 5.1: Communication overhead of vanilla distributed full-graph training (3-layer GCN); "xM-yD" means x machines with y GPUs each [2].

Dataset	Setting	Comm. cost	Remote-neighbour ratio
Reddit	2M-1D	66.78%	41.54%
Reddit	2M-2D	75.20%	62.60%
ogbn-products	2M-2D	75.59%	31.09%
ogbn-products	2M-4D	76.67%	40.52%
AmazonProducts	2M-2D	75.58%	39.75%
AmazonProducts	2M-4D	78.22%	53.00%

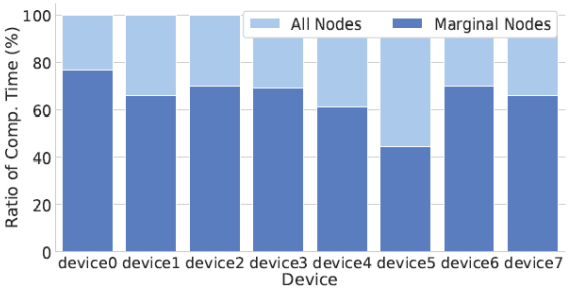
5.1.2 Two Inefficiencies Vanilla Leaves on the Table

AdaQP observes two specific inefficiencies:

Unbalanced all-to-all traffic. METIS balances node count but not communication volume, so different device pairs exchange very different amounts of data. Figure 5.2a shows the transferred data volume per device pair when training GCN on AmazonProducts with 4 partitions, revealing a factor of 2+ spread. In synchronous training this means each communication round waits for its slowest link.



(a) Per-device-pair transferred data size in GCN's first layer on AmazonProducts with 4 partitions.



(b) Ratio of computation time of all nodes vs. marginal nodes only when training on Ogbn-products with 8 partitions; central-node compute, hidden inside the marginal-node communication slot, removes 23%–55% of model compute time.

Figure 5.2: Two sources of waste in vanilla distributed full-graph training: (a) skewed per-device-pair transfer volumes, and (b) the gap between full-graph and marginal-only computation that exposes free compute hidden inside the communication slot.

Central nodes waste compute time. A partition owned by a device can be split into *central* nodes (all of whose neighbours are local) and *marginal* nodes (at least one remote neighbour). The computation of central nodes does not depend on any message exchange and can, in principle, run *during* the message exchange for marginal nodes. Even with messages quantised to 2-bit, marginal-node communication time (0.08–0.13 s per device) is still strictly greater than central-node computation time (0.04–0.06 s per device). This means central-node computation fits inside the communication slot: it is free compute.

Previous efforts either skip messages altogether (SANCUS uses historical embeddings) or pipeline communication with computation using *stale* messages (PipeGCN), which degrades convergence and increases wall-clock time to reach the same accuracy. AdaQP's claim is that compressing what is sent and overlapping with central-node compute can give both speed and accuracy, without staleness.

5.2 Design

5.2.1 System Overview

Each device's partition is decomposed into a central sub-graph (central nodes and their neighbours) and a marginal sub-graph (marginal nodes and their local neighbours). In every GNN layer's forward and backward pass, outgoing marginal messages are *quantised* (with a bit-width decided by the Adaptive Bit-width Assigner), received messages are *de-quantised*, and central-node computation is scheduled to overlap with the marginal-node communication. The Adaptive Bit-width Assigner traces layer inputs online and periodically re-solves a bi-objective optimisation for the next period.

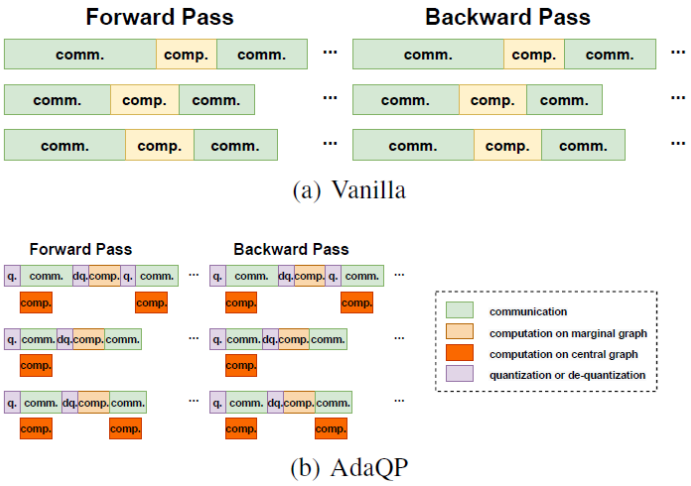


Figure 5.3: Vanilla vs. AdaQP timeline. AdaQP quantises outgoing messages, overlaps central-node computation with marginal-node communication, and de-quantises received messages, removing most of the per-layer wait.

5.2.2 Stochastic Integer Quantisation

For a message vector h_v^l of node v at layer l, AdaQP applies stochastic integer quantisation

$$\tilde{h}_{v,b}^l = \tilde{q}_b(h_v^l) = \text{round}_{st}\left(\frac{h_v^l - Z_v^l}{S_{v,b}^l}\right), \qquad S_{v,b}^l = \frac{\max(h_v^l) - \min(h_v^l)}{2^{b_v} - 1}, \tag{5.1}$$

with $Z_v^l = \min(h_v^l)$, $\text{round}_{st}(\cdot)$ the stochastic rounding operator, and $b_v \in \{2, 4, 8\}$ the chosen bit-width. The receiver de-quantises back to floating point via $\hat{h}_v^l = \tilde{h}_{v,b}^l S_{v,b}^l + Z_v^l$. Key property (Theorem 1 in the paper): $\hat{h}_v^l$ is an unbiased estimate of h_v^l with variance $\text{Var}[\hat{h}_v^l] = D_v^l (S_{v,b}^l)^2/6$.

Because quantisation and de-quantisation are the same linear mapping applied element-wise, AdaQP implements them as simple CUDA kernels; the measured overhead is 5.53%–13.88% of per-epoch training time, which is small compared with the 78.29%–80.94% reduction in communication time.

5.2.3 Central/Marginal Graph Parallelisation

The workflow on each device (Figure 5.4) in each GNN layer is:

1. Marginal nodes: quantise outgoing messages to assigned bit-widths.
2. Marginal nodes: exchange messages across devices (ring-pattern all-to-all); in parallel, central nodes compute (see §5.2.5 for isolation).
3. Marginal nodes: de-quantise received messages.
4. Marginal nodes: compute on the local+remote aggregated messages.

Because central-node compute is hidden inside marginal communication, the model-computation time on each device drops by 23.20%–55.44% (Figure 5.2b). The all-to-all is implemented in a ring pattern (Figure 5.5), which already balances per-round load assuming equal payload but is fragile to the kind of unbalanced volume documented in Figure 5.2a—hence the bi-objective bit-width assignment of the next subsection.

5.2.4 Adaptive Bit-width Assignment

Using a single uniform bit-width is a poor trade-off: 8-bit gives high accuracy but limited compression; 2-bit aggressively compresses but adds variance. AdaQP instead assigns a bit-width per message group, using a 4-step periodic control loop (Figure 5.6):

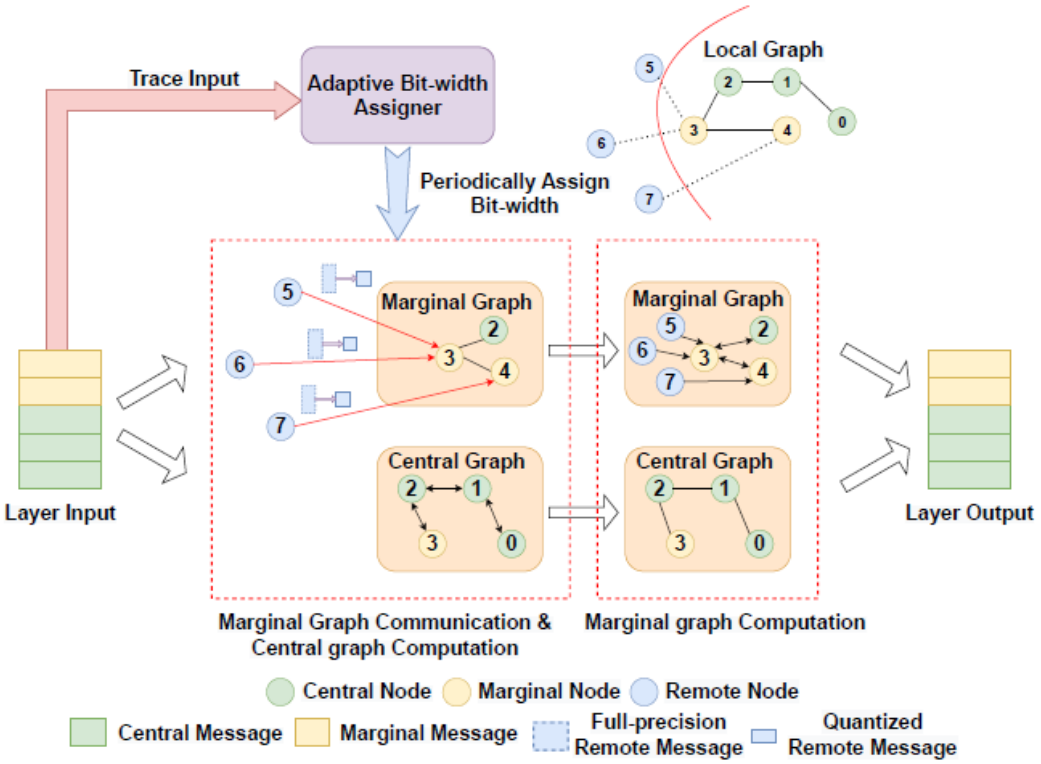


Figure 5.4: Per-layer workflow on each device: central / marginal graph decomposition, marginal-message communication overlapped with central-graph computation, and the Adaptive Bit-width Assigner that periodically updates the per-message bit-width.

1. Each per-device assigner traces the input data of every GNN layer during a bit-width update period.
2. The local assigner sends traced data to a master assigner (on device 0), which builds the bi-objective optimisation problem.
3. The master solves the problem for every layer in parallel (via a thread pool, since layers are independent).
4. The master scatters the new bit-width assignments, and each device updates its sending/receiving buffers.

This control loop turns "pick a bit-width" into a recurring scheduling decision whose state follows the training dynamics.

The Bi-Objective Optimisation

The bi-objective problem trades *straggler communication time* against *gradient variance*. The ring all-to-all uses $N-1$ rounds for N devices; the predicted per-round time on device pair i is $\theta_i \sum_k D_k^l b_k + \gamma_i$ for a measured cost model (θ_i, γ_i). The first objective minimises the slowest pair:

$$\min_{b_k \in B} \max_{1 \le i \le N} \theta_i \sum_{k=1}^{K_i} D_k^l b_k + \gamma_i. \tag{5.2}$$

The second objective minimises gradient variance. Using Theorem 3, the layer-l gradient variance is an additive sum over remote neighbours of terms proportional to $\beta_k/(2^{b_k}-1)^2$, where β_k packs the coefficient sum $\sum_v \alpha_{k,v}^2$, the dimension D_k^l, and the value range:

$$\min_{b_k \in B} \sum_{i=1}^{N} \sum_{k=1}^{K_i} \frac{\beta_k}{(2^{b_k}-1)^2}. \tag{5.3}$$

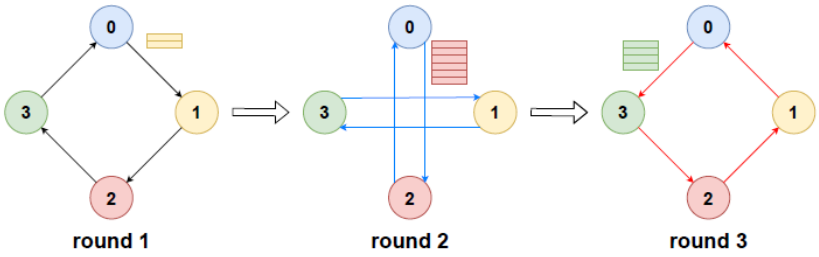


Figure 5.5: Ring all-to-all communication used by AdaQP. For N devices, $N-1$ rounds suffice; in round i, every device exchanges with its i-hop neighbours on the ring.

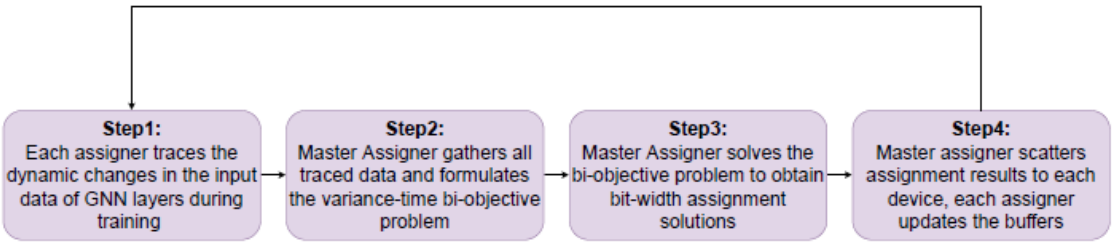


Figure 5.6: Adaptive bit-width assignment process: per-device tracing, master-assigner aggregation, parallel per-layer MILP solve, and scatter back to devices.

Scalarisation with weight $\lambda \in [0, 1]$ and an auxiliary variable Z converts the two objectives into a single Mixed Integer Linear Program:

$$\min_{b_k \in B} \ \lambda \sum_{i,k} \frac{\beta_k}{(2^{b_k}-1)^2} + (1-\lambda)Z, \ \text{ s.t. } \theta_i \sum_k D_k^l b_k + \gamma_i \le Z, \ Z > 0. \tag{5.4}$$

AdaQP views this as an Assignment Problem and solves it with an off-the-shelf MILP solver (GUROBI). To keep the problem small, messages within each device pair are grouped by their β_k value, and all messages in a group share one bit-width.

5.2.5 GPU Resource Isolation

A subtle issue: quantisation, de-quantisation, and central-node compute are *all* compute-intensive and compete with marginal communication for GPU compute/bandwidth resources. If CUDA streams are left to the GPU's own scheduler, the "free" overlap disappears to contention.

AdaQP therefore explicitly controls CUDA kernel launch order (Figure 5.7). The communication-overlap region is split into three fine-grained sub-stages:

1. marginal-graph quantisation (compute-heavy),
2. marginal-graph communication *with* central-graph computation (communication uses bandwidth, central compute uses SMs),
3. marginal-graph de-quantisation (compute-heavy).

Different kernels are wrapped in different CUDA streams, and CPU and GPU events are used to synchronise their launch and completion. Because central-graph compute and marginal communication use *disjoint* GPU resources (SM cycles vs. NVLink/PCIe bandwidth), they can run simultaneously without mutual slowdown.

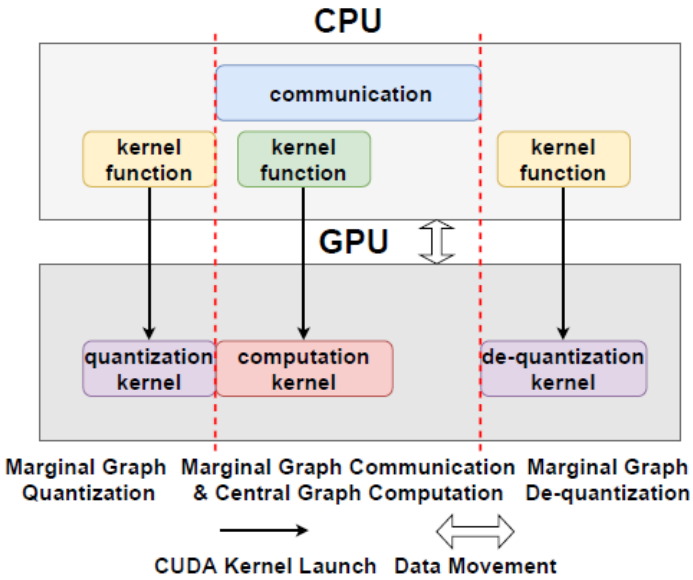


Figure 5.7: GPU resource isolation strategy: quantisation, central-graph computation & marginal communication, and de-quantisation are placed in separate CUDA streams ordered by CPU/GPU events.

5.3 Convergence Guarantee

Because AdaQP uses *lossy* compression, the natural concern is whether the training still converges at the same rate as vanilla full-graph training. The paper proves that it does.

Under standard assumptions (A.1 L_2-Lipschitz gradient; A.2 existence of global minimum; A.3 unbiased stochastic gradient; A.4 bounded variance), Theorem 2 of the paper shows that for a fixed step size $\alpha < 2/L_2$, a randomly chosen iterate $\bar{w}_t$ from T epochs satisfies

$$\mathbb{E}\big[\|\nabla L(\bar{w}_t)\|^2\big] \le \frac{2(L(w_1) - L^*)}{T(2\alpha - \alpha^2 L_2)} + \frac{\alpha L_2 Q^2}{2 - \alpha L_2}. \tag{5.5}$$

The first term decays as $O(T^{-1})$. The variance Q^2 entering the second term is *not* sampling variance (full-graph training has none) but exactly the quantisation variance whose bound Q_l in each layer is given by Theorem 3 as a function of graph topology, partitioning, aggregation coefficients, dimension, value range, and bit-width. Choosing the bit-widths is therefore *equivalent* to choosing the variance budget. AdaQP's bit-width assigner uses this bound explicitly in Eq. (5.3).

The resulting $O(T^{-1})$ rate matches vanilla full-graph training and is strictly better than sampling-based methods ($O(T^{-1/2})$) and staleness-based methods ($O(T^{-2/3})$ for PipeGCN, $O(T^{-1/2})$ for SANCUS).

5.4 Evaluation

5.4.1 Setup

AdaQP is implemented on DGL 0.9 and PyTorch 1.11. Four datasets are used (Table 5.2). Two models (GCN and full-batch GraphSAGE, three layers, 256 hidden) are trained on two servers with 4×V100 each, connected by 100 Gbps Ethernet. Partition settings range from 2M-1D (2 machines, 1 GPU each) to 2M-4D (2 machines, 4 GPUs each). Baselines are Vanilla, PipeGCN, and SANCUS.

Table 5.2: Datasets used by AdaQP [2].

Dataset	#Nodes	#Edges	#Feat.	#Class	Size
Reddit	232,965	114,615,892	602	41	3.53 GB
Yelp	716,847	6,977,410	300	100	2.10 GB
ogbn-products	2,449,029	61,859,140	100	47	1.38 GB
AmazonProducts	1,569,960	264,339,468	200	107	2.40 GB

5.4.2 Throughput and Accuracy

Across the 16 (dataset, partition, model) cells, AdaQP wins the highest throughput in 14 cells and the highest accuracy in 12 cells (selected results in Table 5.3). Headline numbers: $2.19\times$ to $3.01\times$ speedup over Vanilla, with accuracy changes between -0.30% and $+0.19\%$. By contrast, PipeGCN incurs up to 3.8% accuracy loss on AmazonProducts, and SANCUS drops several percentage points on multi-label datasets. The slight accuracy *gain* of AdaQP in some settings is attributed to the regularisation effect of quantisation noise.

Table 5.3: Selected end-to-end results: accuracy (%) and throughput (epoch/s) for GCN and GraphSAGE, AdaQP vs. Vanilla, on 2M-2D and 2M-4D [2]. "†" marks settings where the baseline is not implemented for that model. Speedup of AdaQP over Vanilla is shown in parentheses.

Dataset	Setting	Vanilla		AdaQP	
		Acc. (%)	Thr.	Acc. (%)	Thr.
Reddit (GCN)	2M-2D	95.35	1.13	95.38	2.65 (2.35×)
Reddit (GraphSAGE)	2M-2D	96.55	1.16	96.53	2.65 (2.28×)
Yelp (GCN)	2M-2D	43.86	1.57	43.84	3.64 (2.32×)
Yelp (GraphSAGE)	2M-2D	64.67	1.19	64.78	3.58 (3.01×)
ogbn-products (GCN)	2M-4D	75.11	0.79	75.30	2.18 (2.76×)
ogbn-products (SAGE)	2M-4D	78.89	0.77	78.90	2.15 (2.79×)
AmazonProducts (GCN)	2M-4D	51.38	0.58	51.56	1.60 (2.76×)
AmazonProducts (SAGE)	2M-4D	75.80	0.62	75.98	1.61 (2.60×)

One exception: PipeGCN is faster than AdaQP on Reddit, because Reddit is exceptionally dense and PipeGCN's cross-iteration pipeline hides the (already small) communication time well on dense graphs. On sparse graphs this property does not hold and PipeGCN loses ground.

5.4.3 Preserved Convergence Rate

Figure 5.8 plots validation accuracy vs. epoch for all methods. AdaQP's curves nearly coincide with Vanilla's on every dataset/setting, empirically confirming the $O(T^{-1})$ analysis. PipeGCN and SANCUS need more epochs to reach the same accuracy. On AmazonProducts the wall-clock picture (Table 5.4) is striking: AdaQP finishes in $\sim$ 770–1050 s versus $\sim$ 1930–2900 s for Vanilla, $\sim$ 3800 s for SANCUS (GCN), and $\sim$ 1170 s for PipeGCN (GraphSAGE).

Table 5.4: Wall-clock time (seconds) on AmazonProducts, AdaQP vs. the three baselines [2]. "†" marks settings where the baseline is not implemented for that model.

Setting	Model	Vanilla	PipeGCN	SANCUS	AdaQP
2M-2D	GCN	2874.77	†	3782.44	**1053.51**
2M-2D	GraphSAGE	2597.21	1212.65	†	**1008.34**
2M-4D	GCN	2057.70	†	3880.68	**806.29**
2M-4D	GraphSAGE	1927.85	1171.38	†	**771.52**

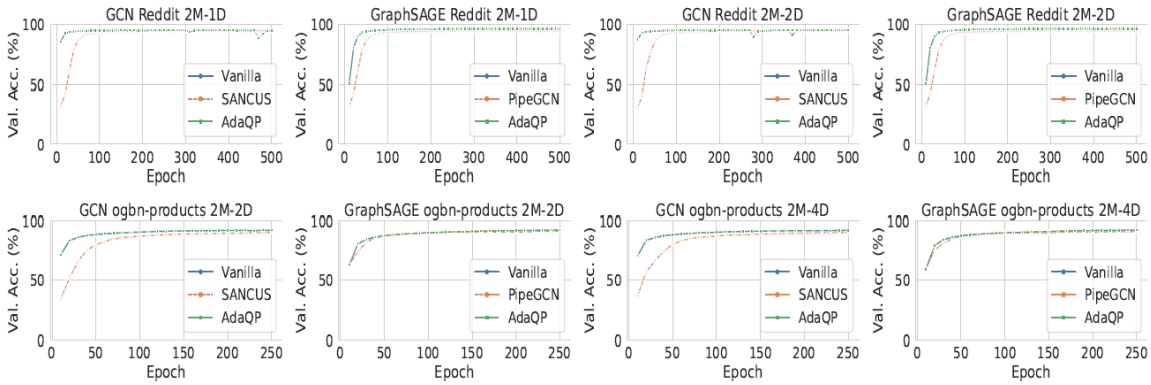


Figure 5.8: Validation accuracy vs. epoch on Reddit and Ogbn-products for Vanilla, PipeGCN, SANCUS and AdaQP.

5.4.4 Adaptive vs. Uniform Bit-width

A natural ablation is "what if we just pick a random bit-width from $\{2, 4, 8\}$ per message?". Table 5.5 compares this uniform sampling against AdaQP's adaptive scheme on Ogbn-products. Adaptive matches or slightly exceeds uniform in throughput while beating it by up to 0.29% accuracy (e.g., 75.32% vs. 75.03%). The reason is that uniform sampling will occasionally assign 2-bit to a message with a large β value, which injects large gradient variance. Adaptive assignment, guided by Eq. (5.3), prefers more bits for such messages and fewer bits for messages that can afford them.

Table 5.5: Adaptive vs. uniform bit-width sampling on Ogbn-products [2].

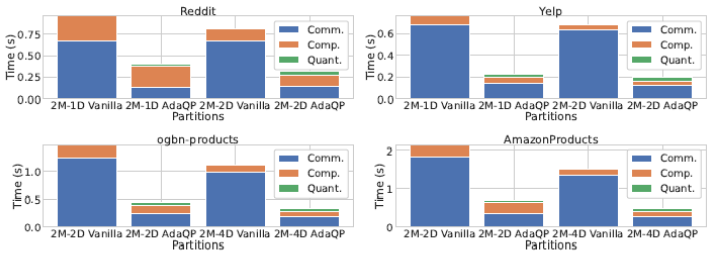
Setting	Model	Uniform		Adaptive	
		Acc. (%)	Thr.	Acc. (%)	Thr.
2M-2D	GCN	75.03	1.70	**75.32**	1.65
2M-2D	GraphSAGE	78.84	1.64	**78.85**	1.67
2M-4D	GCN	75.16	2.14	**75.30**	2.18
2M-4D	GraphSAGE	78.85	2.07	**78.90**	2.15

5.4.5 Time Breakdown and Overhead Accounting

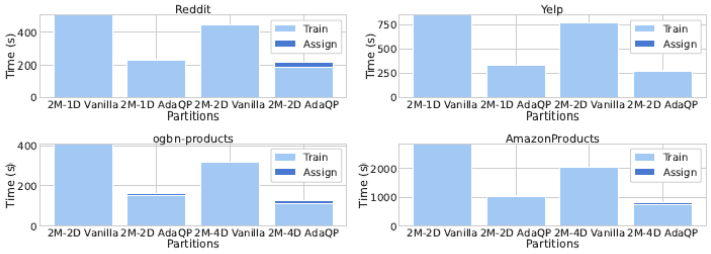
The paper decomposes per-epoch time into communication, computation, and quantisation. Across datasets, quantisation adds 5.53%–13.88% of per-epoch time; in exchange, communication time drops by 78.29% and computation time drops by 23%–55% (because central compute is hidden). The net throughput gain is therefore dominated by saved communication and hidden central-compute, and the assigner's overhead is small enough to amortise across many epochs.

5.5 Summary

AdaQP represents the "communicate less and wait less" line of OS support for graph learning. Its contribution is to combine two classically OS-flavoured techniques—*lossy message compression* (stochastic integer quantisation with an adaptive per-message bit-width chosen by an online MILP) and *resource-isolated scheduling* (CUDA-stream-level isolation of quantise / communicate+compute / de-quantise)—into a controlled distributed training workflow. A formal convergence analysis proves the resulting system retains vanilla's $O(T^{-1})$ rate; experiments show $2.19\times$–$3.01\times$ throughput improvement with $\leq 0.30\%$ accuracy impact on four real graph datasets, outperforming both vanilla and staleness-based state-of-the-art baselines.



(a) Per-epoch time



(b) Wall-clock time

Figure 5.9: Per-epoch time breakdown (communication / computation / quantisation) of Vanilla and AdaQP across datasets and partition settings, plus wall-clock split into bit-width assignment vs. actual training.

Chapter 6

Conclusion and Discussion

6.1 Cross-Paper Comparison

The three papers attack three different stages of the GNN training pipeline, and therefore make three different design choices. Table 6.1 summarises the comparison.

Table 6.1: Cross-paper comparison of BGL, MGG, and AdaQP.

	BGL [1]	**MGG** [3]	**AdaQP** [2]
Training regime	Sampling-based distributed	Single-node multi-GPU full-graph	Distributed full-graph
Main bottleneck	Data I/O and preprocessing (80–90% of iteration time)	Remote access on the critical path of irregular kernels	All-to-all communication volume, imbalance and stragglers ($\sim$ 66–78% of epoch time)
Main system idea	Dynamic FIFO cache with proximity-aware ordering; GNN-aware partitioning; profiled resource isolation	Three-stage LR/LL/AC intra-kernel software pipeline; warp-based mapping with interleaving; analytical-model-driven runtime	Adaptive stochastic-integer message quantization (MILP-chosen bit-widths); central/marginal graph parallelisation; explicit CUDA-stream resource isolation
Headline results	1.14–69$\times$ vs. four baselines; geom. mean 1.91$\times$ over PaGraph; GPU utilisation up to 99% (GAT) and 65% (GraphSAGE)	4.25$\times$ / 4.57$\times$ average over DGL on GCN / GIN; 4.58$\times$ / 5.04$\times$ over MGG-UVM; 12.3$\times$ / 9.35$\times$ over ROC; up to 68% latency saved by runtime search	2.19–3.01$\times$ throughput over Vanilla; up to 80.94% communication-time reduction; accuracy -0.30% to $+0.19\%$; provable $O(T^{-1})$ convergence
Accuracy stance	No loss (same as DGL)	No loss (deterministic execution)	Near-lossless (quantisation noise has a small regularisation effect)

6.2 A Storyline that Moves Outward Along the Stack

Read together, the three papers form a coherent storyline rather than three isolated tricks. The optimisation target moves gradually *outward* along the system stack:

- **BGL** optimises how the system *prepares* training data for the GPU.
- **MGG** optimises how a multi-GPU node *executes* the irregular GNN kernel.
- **AdaQP** optimises *what* is exchanged across devices and *when* computation waits.

This is also the natural order in which bottlenecks appear as a GNN training system scales from a single GPU to a multi-GPU node and finally to many distributed nodes: first the front-end starves the GPU (fix it with BGL); then inside a node the irregular kernel wastes SM cycles on remote-access latency (fix it with MGG); then across nodes the synchronous all-to-all becomes the critical path (fix it with AdaQP).

6.3 An OS Perspective

From the point of view of an operating-systems course, the three systems are striking for how many classical OS ideas they recycle.

- *Caching with locality-aware replacement policies.* BGL's FIFO+proximity cache is a close cousin of page replacement under a known access pattern: trading a weaker policy (FIFO beats LRU/LFU on update cost) for a stronger access order (BFS traversal of training nodes concentrates reuse in the FIFO's recency window).
- *Multi-level partitioning under multiple constraints.* BGL's block coarsening plus assignment heuristic and MGG's local/remote neighbour split echo the classical combination of locality-driven and balance-driven partitioning seen in multicore graph processing.
- *Pipeline construction with scheduled stages.* MGG's three-stage LR/LL/AC pipeline with explicit interleaving distance is the GPU analogue of pipelined execution with configurable depth; AdaQP's ordered stages "quantise $\rightarrow$ communicate+compute $\rightarrow$ de-quantise" is the same idea at distributed-system level.
- *Lossy encoding plus scheduler-level prioritisation.* AdaQP's adaptive quantisation combined with CUDA-stream isolation is the distributed-systems analogue of compressed I/O combined with priority scheduling. The variance-time bi-objective optimisation is the quality knob on the compressor.
- *Profiling-driven resource allocation.* BGL's min-max of per-stage completion times subject to CPU/PCIe caps, and MGG's lookup-table-driven parameter search, are both instances of the classical OS idea of re-planning allocation from observed load.

The key lesson is that the GNN systems literature is not inventing new OS primitives; it is re-applying well-understood OS primitives under the very specific constraints of irregular, communication-heavy graph workloads.

6.4 Concluding Remarks

Large-scale graph learning stresses systems because its bottlenecks *move* between data preparation, irregular compute, and inter-device communication as the workload scales. BGL shows that OS-style pipeline design and locality management can fix the front end of the training loop. MGG shows that inside a multi-GPU node, fine-grained overlap and runtime-aware kernel mapping matter a great deal. AdaQP shows that in distributed full-graph training, reducing message volume and hiding wait time can improve throughput without a large accuracy loss and without relying on stale messages. Taken together, they make the case that the path to fast large-scale GNN training is not a new algorithmic trick but a carefully engineered operating-system layer on top of existing deep-learning frameworks.

Bibliography

[1] Tianfeng Liu, Yangrui Chen, Dan Li, Chuan Wu, Yibo Zhu, Jun He, Yanghua Peng, Hongzheng Chen, Hongzhi Chen, and Chuanxiong Guo. BGL: GPU-Efficient GNN Training by Optimizing Graph Data I/O and Preprocessing. In *Proc. of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[2] Borui Wan, Juntao Zhao, and Chuan Wu. Adaptive Message Quantization and Parallelization for Distributed Full-Graph GNN Training. In *Proc. of Machine Learning and Systems (MLSys)*, 2023. arXiv preprint arXiv:2306.01381.

[3] Yuke Wang, Boyuan Feng, Zheng Wang, Tong Geng, Kevin Barker, Ang Li, and Yufei Ding. MGG: Accelerating Graph Neural Networks with Fine-Grained Intra-Kernel Communication-Computation Pipelining on Multi-GPU Platforms. In *Proc. of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023. arXiv preprint arXiv:2209.06800.